



Week 2

Machine Learning Introduction

AUTHOR

MAKSYM BONDAR

25.11.2025

Kaunas

Contents

1	Unsupervised Learning: Principal Component Analysis and Clustering	1
1.1	Dimensionality Reduction and Clustering Models	1
1.1.1	Principal Component Analysis (PCA)	1
1.1.2	k-means Clustering	2
1.1.3	Agglomerative (Hierarchical) Clustering	3
1.1.4	Spectral Clustering	4
1.1.5	Affinity Propagation	5
1.1.6	Clustering Metrics and Comparison	5
1.1.7	Supervised Baseline: Linear SVM	6
2	Some aspects of ML	8
2.1	Stochastic Gradient Descent for Linear Regression	8
2.1.1	Problem Setup	8
2.1.2	Model Implementation	9
2.1.3	Training Dynamics and Convergence	10
2.1.4	Comparison with Ordinary Least Squares	11
3	Some aspects of ML	13
3.1	Preparing the Time Series	13
3.2	Moving Average and Rolling Smoothing	13
3.3	Exponential Smoothing	15
3.4	Double Exponential Smoothing (Holt’s Method)	16
3.5	Triple Exponential Smoothing (Holt–Winters)	17
3.6	Time Series Cross-Validation	17
3.7	Forecasting and Anomaly Detection	18
3.8	Stationarity and SARIMA Modelling	20
3.9	Linear Models for Time Series	26
3.10	Regularization and Feature Selection: Ridge and Lasso	33
3.11	Gradient Boosting for Time Series (XGBoost)	36
4	Gradient Boosting	39
4.1	Gradient Boosting for Bike Rental Forecasting	39
	References	43

1 Unsupervised Learning: Principal Component Analysis and Clustering

1.1 Dimensionality Reduction and Clustering Models

This section describes the theoretical motivation behind the models used in the analysis, together with the key results obtained from applying them to the daily bike-sharing dataset. The goal was to understand whether days with similar demand levels form natural clusters and whether those structures can be revealed after projecting the data into a lower-dimensional space.

1.1.1 Principal Component Analysis (PCA)

The dataset contains several correlated features such as temperature, perceived temperature, humidity, or season indicators. Principal Component Analysis (PCA) provides a way to replace these original variables with a smaller number of uncorrelated components that capture most of the total variance.

Mathematically, PCA seeks eigenvectors of the covariance matrix:

$$Sv_i = \lambda_i v_i, \quad S = \frac{1}{n-1} X^\top X,$$

where the eigenvectors v_i define the new orthogonal axes and the eigenvalues λ_i represent the variance explained by each component.

In our experiment, retaining enough components to explain 90% of the variance resulted in an 8-dimensional representation:

Explained variance ratio (sum) = 0.9562.

The first component alone explained approximately:

24.3% of total variance.

```
1 pca = PCA(n_components=0.90, random_state=RANDOM_STATE)
2 X_pca = pca.fit_transform(X_scaled)
3 pca.explained_variance_ratio_
```

A 2D visualization of the first two principal components is shown in Figure 1, with points colored by the true demand level (low, medium, high).

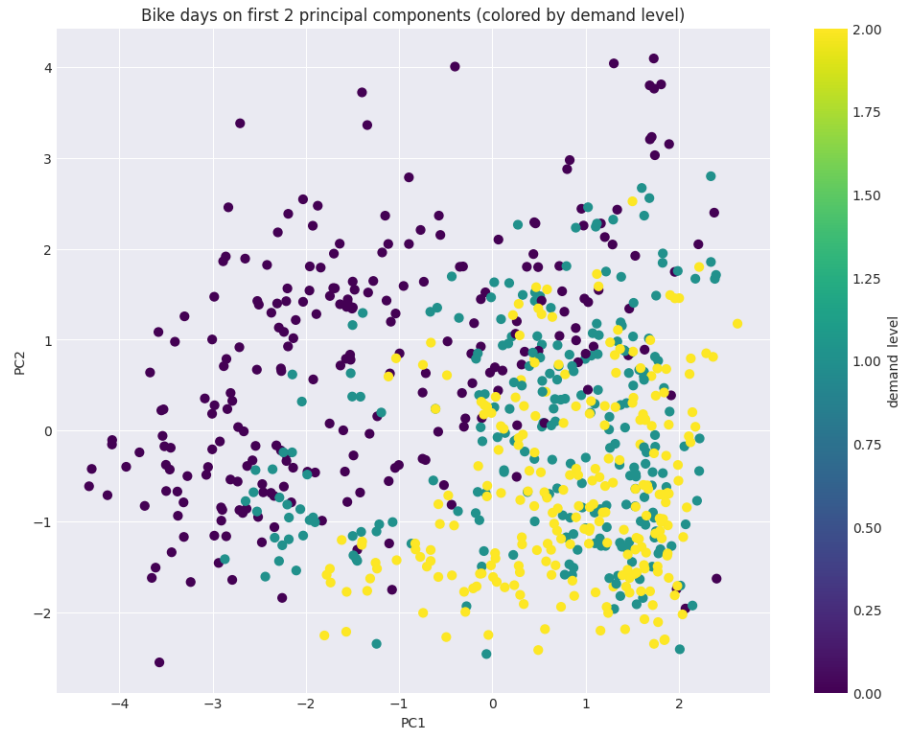


Figure 1: Bike-sharing days projected on the first two principal components.

1.1.2 k-means Clustering

k-means is one of the simplest approaches to partition a dataset. It alternates between assigning each point to the closest centroid and updating centroids to minimize within-cluster variance:

$$J = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2.$$

In this study, the algorithm was applied to the PCA-transformed data with $k = 3$ clusters, corresponding to the three defined demand levels.

```
1 kmeans = KMeans(n_clusters=3, n_init=100)
2 cluster_labels_kmeans = kmeans.fit_predict(X_pca)
```

The resulting clusters are illustrated in Figure 2.

The contingency table comparing true demand levels and k-means clusters revealed moderate alignment:

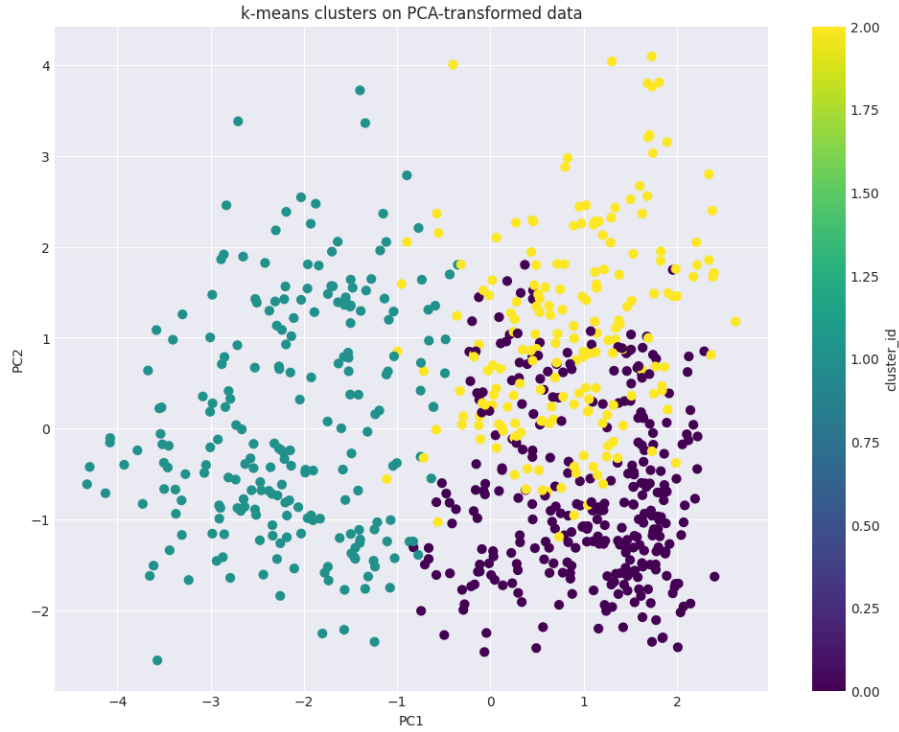


Figure 2: k-means clustering on PCA-transformed bike data.

	C_0	C_1	C_2
low	16	160	68
medium	131	46	67
high	162	22	59

A simple measure of separability (share of each demand level captured by its dominant cluster) was:

low: 0.656, medium: 0.537, high: 0.667.

Finally, an elbow curve (Figure 3) illustrated how inertia decreases with increasing k .

1.1.3 Agglomerative (Hierarchical) Clustering

Hierarchical clustering begins with each point as its own cluster and iteratively merges the closest pairs. Using Ward linkage ensures that each merge minimally increases total within-cluster variance:

$$\Delta(C_i, C_j) = \frac{|C_i||C_j|}{|C_i| + |C_j|} \|\mu_i - \mu_j\|^2.$$

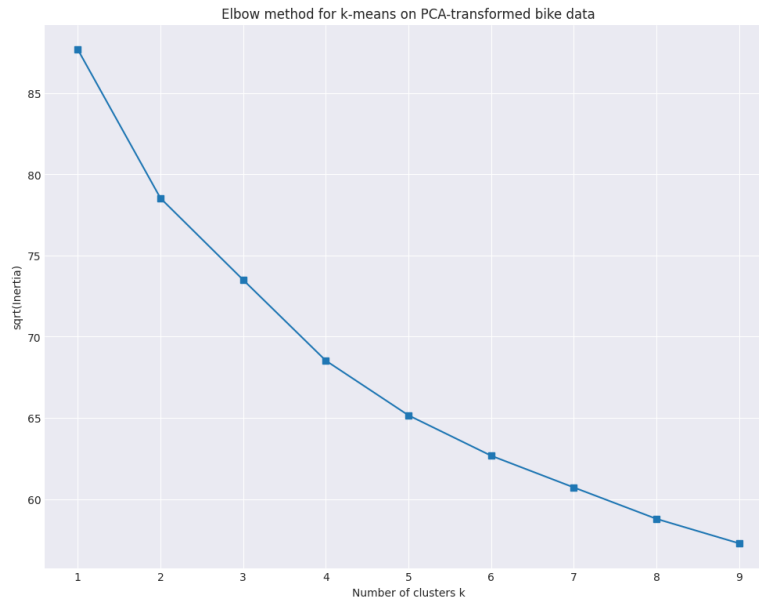


Figure 3: Elbow method for selecting the number of k-means clusters.

```
1 ag = AgglomerativeClustering(n_clusters=3, linkage="ward")
2 cluster_labels_ag = ag.fit_predict(X_pca)
```

The resulting clusters (Figure 4) show a different structure from k-means but similar overall behavior.

1.1.4 Spectral Clustering

Spectral clustering constructs a similarity graph of the data and analyzes the eigenvectors of its Laplacian matrix:

$$L = D - W,$$

where W is the similarity matrix and D the degree matrix.

Instead of clustering in the original space, it clusters in this transformed eigenvector space, which often reveals non-linear or curved cluster structures.

```
1 spec = SpectralClustering(n_clusters=3, assign_labels="kmeans")
2 cluster_labels_spec = spec.fit_predict(X_pca)
```

The result is displayed in Figure 5.



Figure 4: Agglomerative clustering with Ward linkage.

1.1.5 Affinity Propagation

Affinity Propagation does not require specifying the number of clusters beforehand. Instead, it identifies exemplar points that best summarize the dataset through a message-passing mechanism involving responsibilities and availabilities.

```
1 aff = AffinityPropagation()  
2 cluster_labels_aff = aff.fit_predict(X_pca)
```

In this dataset AP produced a relatively large number of clusters (Figure 6).

1.1.6 Clustering Metrics and Comparison

Three metrics were used to evaluate clustering quality:

- Adjusted Rand Index (ARI): measures agreement with true labels, corrected for chance.
- Adjusted Mutual Information (AMI): measures shared information between cluster labels and true labels.
- Silhouette Score: measures cohesion and separation without using true labels.

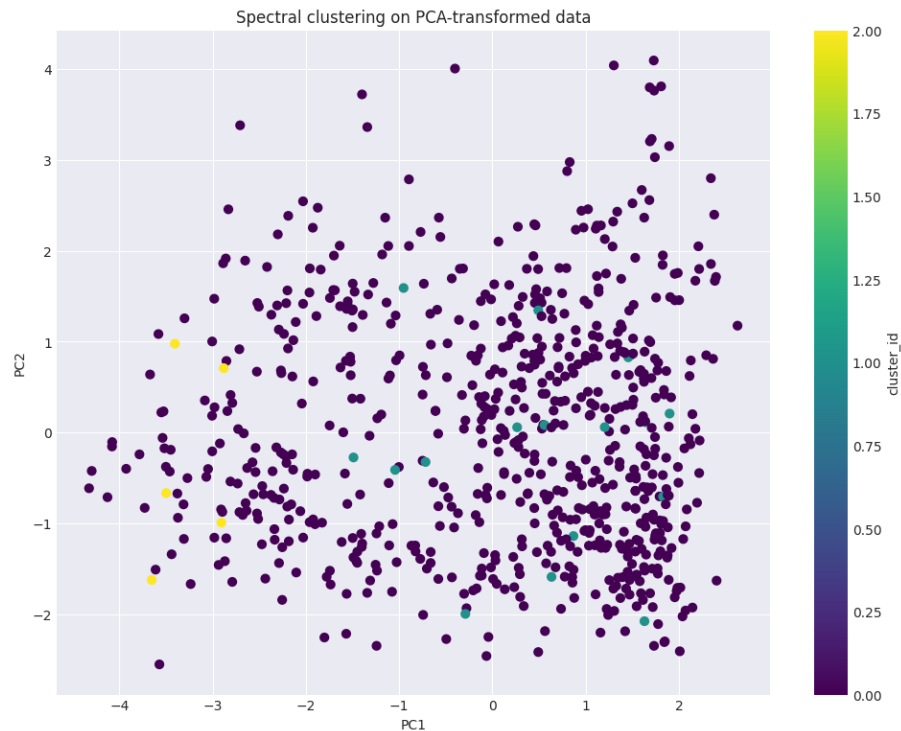


Figure 5: Spectral clustering on PCA-transformed data.

The scores obtained were:

Method	ARI	AMI	Silhouette
k-means	0.1876	0.1791	0.1922
Agglomerative	0.1210	0.1237	0.2072
Affinity	0.0482	0.2332	0.2264
Spectral	0.0003	0.0114	0.4094

Interestingly, Spectral clustering achieved the highest silhouette score, despite poor ARI and AMI, suggesting that Spectral captured some structural separation in the PCA space that did not correspond directly to the defined demand levels.

1.1.7 Supervised Baseline: Linear SVM

To provide a reference for how well demand levels can be predicted with labeled data, a Linear SVM classifier was trained both with and without PCA.

```

1 svc = LinearSVC()
2 grid = GridSearchCV(svc, {"C":[0.001,0.01,0.1,1,10]}, cv=3)
3 grid.fit(X_train_scaled, y_train)

```

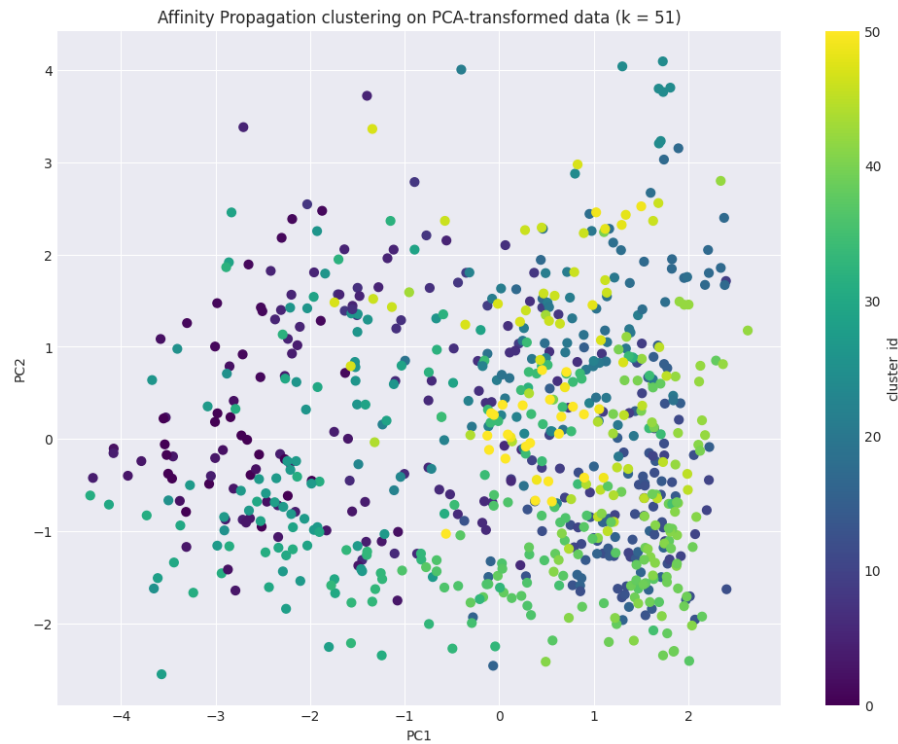


Figure 6: Affinity Propagation clustering results.

Results were:

Test accuracy without PCA: 0.795.

Test accuracy with PCA: 0.800.

Both versions produced similar performance and significantly outperformed unsupervised clustering, as expected. Nonetheless, clustering provided valuable insight into underlying structure and variability in bike-sharing behavior.

2 Some aspects of ML

2.1 Stochastic Gradient Descent for Linear Regression

In this part of the analysis, the goal was to revisit one of the core ideas from the course: training a linear regression model using Stochastic Gradient Descent (SGD). Instead of solving the normal equations in closed form, SGD updates the weights iteratively after observing each individual training example. This makes the algorithm scalable and capable of online adaptation, although at the cost of higher variance and potentially slower convergence.

2.1.1 Problem Setup

We focus on a simple univariate regression problem: predicting the total number of bike rentals (cnt) based solely on normalized temperature (temp), a variable known to correlate strongly with user demand.

Figure 7 displays the raw scatter plot of temperature vs. total daily rentals.

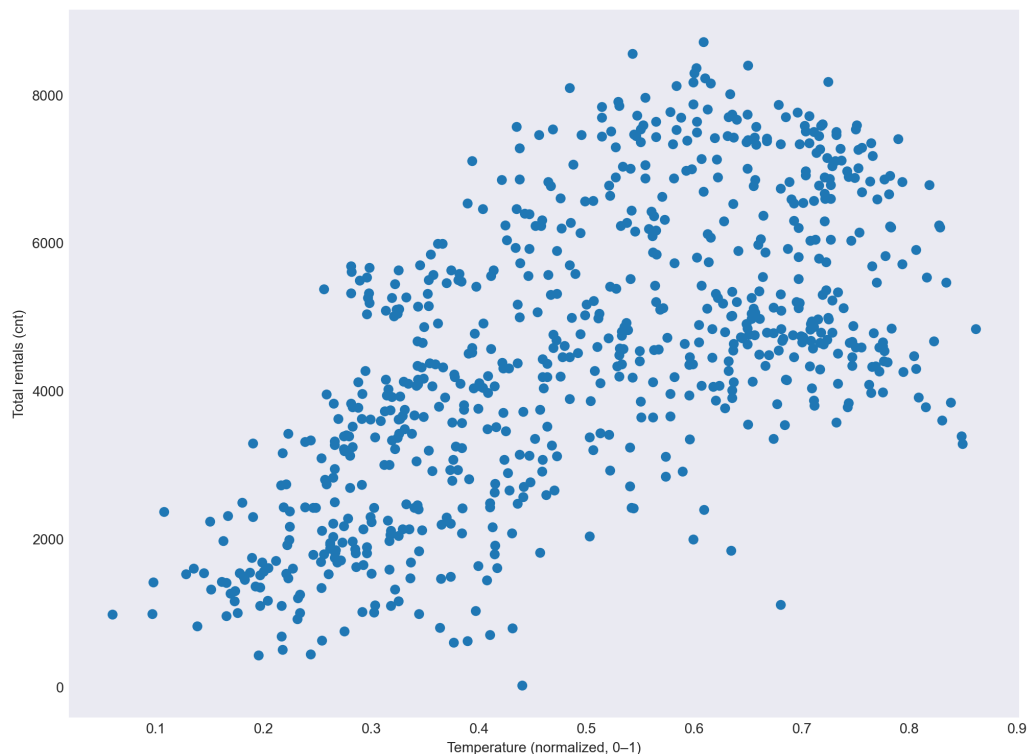


Figure 7: Scatter plot of temperature (0–1 normalized) and total bike rentals.

Formally, we seek a linear model

$$\hat{y} = w_0 + w_1x,$$

where x is the scaled temperature and y is the number of rentals.

Training is performed by minimizing the Mean Squared Error (MSE):

$$MSE(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2.$$

In Stochastic Gradient Descent, we update the parameters after each example:

$$\begin{aligned} w_0 &\leftarrow w_0 + \eta(y_i - \hat{y}_i), \\ w_1 &\leftarrow w_1 + \eta(y_i - \hat{y}_i)x_i, \end{aligned}$$

where η is the learning rate.

2.1.2 Model Implementation

The implementation follows the exact structure used in the course assignment. The regressor records both the MSE and the evolving weight vector at each update step.

```
1 class SGDRegressor(BaseEstimator):
2     def __init__(self, eta=1e-3, n_epochs=3):
3         self.eta, self.n_epochs = eta, n_epochs
4         self.mse_, self.weights_ = [], []
5
6     def fit(self, X, y):
7         X = np.hstack([np.ones([X.shape[0], 1]), X])
8         w = np.zeros(X.shape[1])
9
10        for _ in range(self.n_epochs):
11            for i in range(X.shape[0]):
12                err = y[i] - w.dot(X[i])
13                w[0] += self.eta * err
14                w[1] += self.eta * err * X[i,1]
15
16                self.weights_.append(w.copy())
17                self.mse_.append(mean_squared_error(y, X.dot(w)))
18
19        self.w_ = self.weights_[np.argmin(self.mse_)]
20        return self
```

The training used:

$$X = \text{temp}, \quad y = \text{cnt},$$

with standard scaling applied to temperature.

2.1.3 Training Dynamics and Convergence

The evolution of the Mean Squared Error across all SGD updates is shown in Figure 8. The curve decreases steadily but exhibits the characteristic noisy shape expected from stochastic updates.

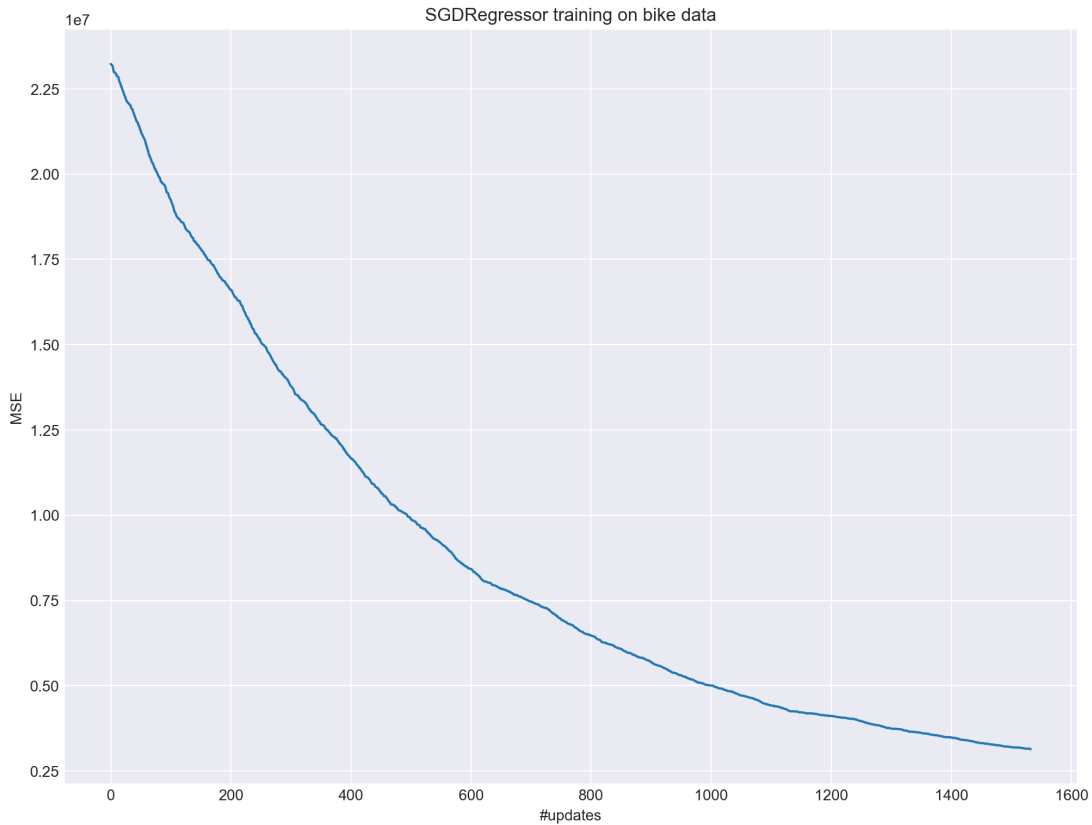


Figure 8: MSE during training of the SGDRegressor.

The minimum MSE achieved during training was:

$$\min MSE = 3.1385 \times 10^6.$$

The corresponding learned weights were:

$$w_0 = 3470.22, \quad w_1 = 981.75.$$

To illustrate the dynamics, Figure 9 shows the trajectories of the bias and the slope parameters across all updates.

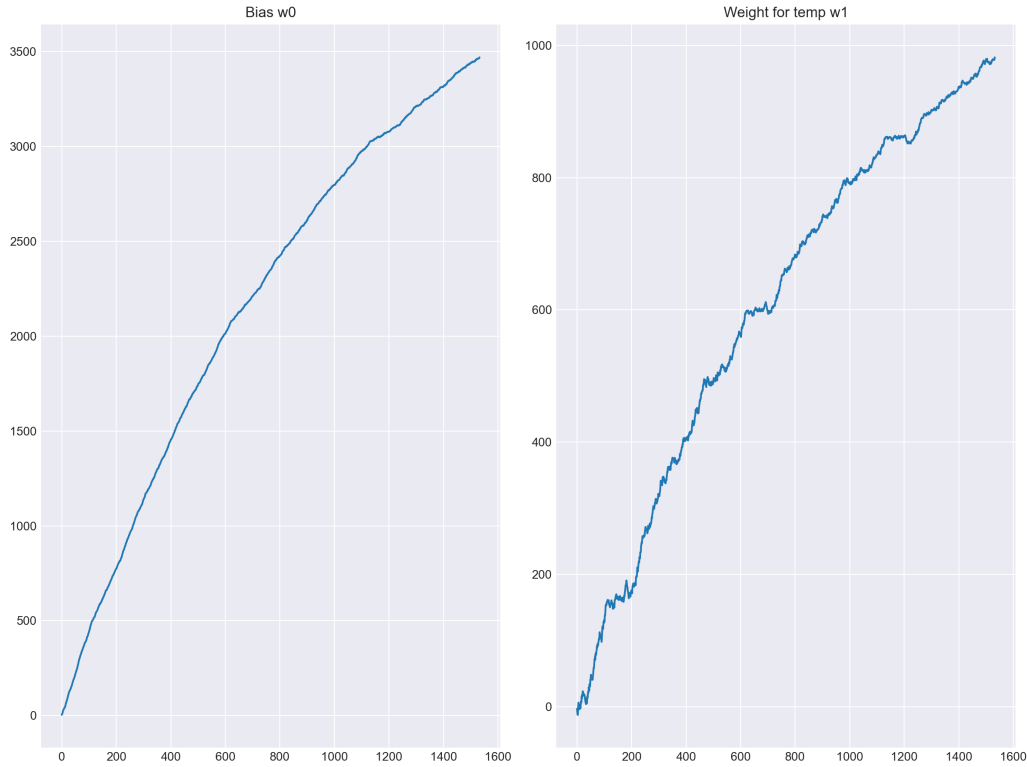


Figure 9: Evolution of weights during SGD training. Left: bias w_0 . Right: temperature coefficient w_1 .

2.1.4 Comparison with Ordinary Least Squares

To evaluate accuracy, we computed the hold-out MSE using a standard `LinearRegression` model for comparison.

The linear regression solution achieved:

$$\text{LinearRegression MSE} = 2.5181 \times 10^6,$$

with parameters:

$$w_0 = 4419.80, \quad w_1 = 1244.54.$$

The SGD-based solution performed worse on the holdout set:

$$\text{SGDRegressor MSE} = 3.5157 \times 10^6.$$

The performance comparison is presented in Table 1.

With only three epochs and a fixed learning rate, SGD converges slowly and remains far from the optimal OLS solution. The MSE curve is monotonically decreasing, meaning learning is progressing, but insufficient training or a poorly tuned η prevents reaching the true minimum.

The gap between the OLS and SGD solutions illustrates key theoretical properties of SGD:

Table 1: Hold-out MSE comparison between SGD and closed-form Linear Regression.

Model	Hold-out MSE	Notes
Linear Regression (OLS)	2.5181×10^6	Optimal closed-form solution
SGDRegressor	3.5157×10^6	Suboptimal due to fixed η and few epochs

- SGD explores the error landscape one example at a time, leading to noisy but cheap updates.
- With more epochs or a learning rate schedule (e.g. decreasing η), SGD would approach the OLS optimum.
- SGD is useful for large datasets where OLS becomes computationally expensive but for this small dataset, OLS is strictly superior.

Overall, this experiment validates the course material: SGD is effective and interpretable, but sensitive to hyperparameters and usually requires more thorough tuning to match closed-form regression performance.

3 Some aspects of ML

3.1 Preparing the Time Series

To begin the time series analysis, the original bike rentals dataset was transformed into a daily time-indexed series. The column `dteday` was converted into a `datetime` object and set as the index:

```
1 df["dteday"] = pd.to_datetime(df["dteday"])
2 df = df.sort_values("dteday").set_index("dteday")
3 ts = df[["cnt"]].asfreq("D")
```

The resulting series spans from January 2011 to January 2013 and represents the total number of rentals per day. Figure 10 shows the daily “cnt” time series.

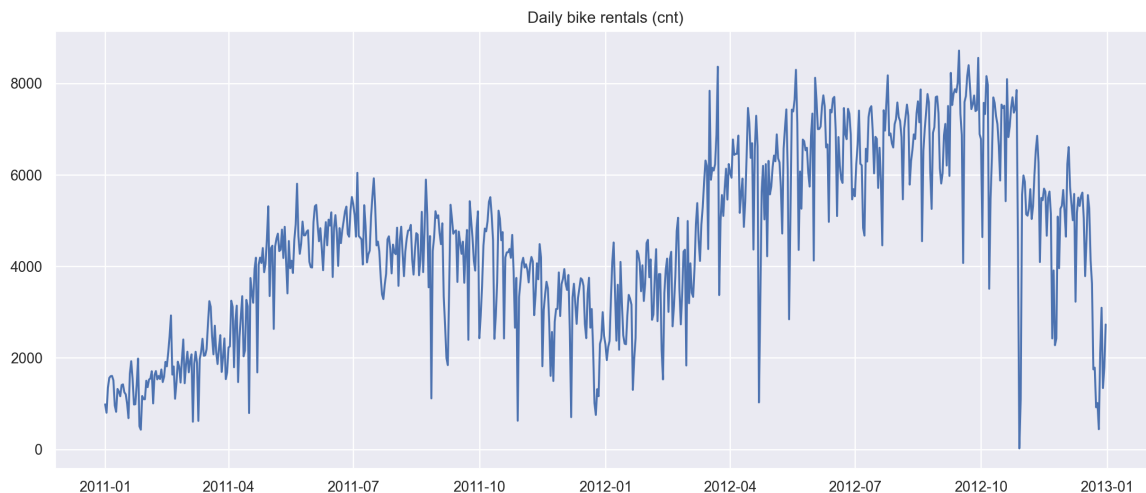


Figure 10: Daily bike rentals from 2011 to 2012

We will study this series using smoothing techniques, Holt Winters models, and time series cross-validation to produce a 30-day forecast.

3.2 Moving Average and Rolling Smoothing

A simple yet powerful way to extract the underlying trend is through the moving average:

$$MA_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} y_{t-i}.$$

A helper function was implemented:

```
1 def moving_average(series, n):  
2     return np.average(series[-n:])
```

For example, the average rentals in the last 7 days was:

$$MA_7 = 1789.86.$$

To visualize the moving average and its uncertainty, a rolling window approach was used:

```
1 plotMovingAverage(ts["cnt"], 7)      # weekly  
2 plotMovingAverage(ts["cnt"], 30)    # ~monthly  
3 plotMovingAverage(ts["cnt"], 90)    # long-term
```

Figures 11–13 illustrate the smoothing levels.

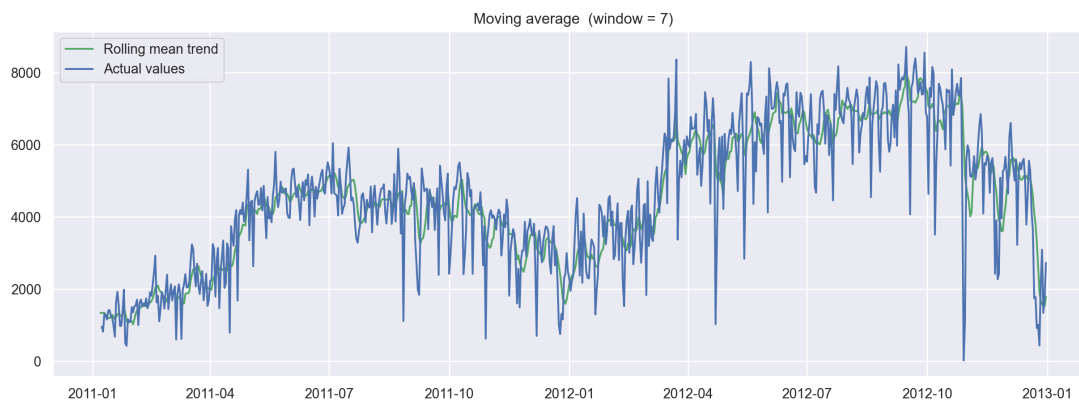


Figure 11: 7-day moving average

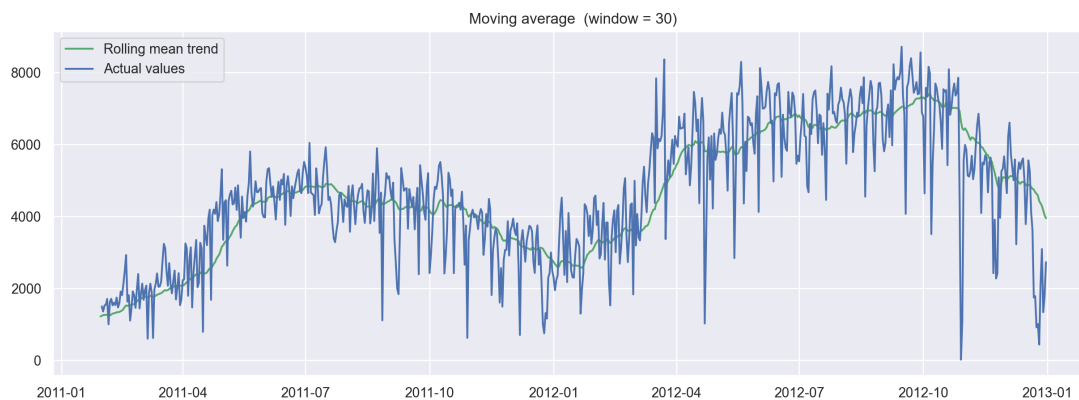


Figure 12: 30-day moving average

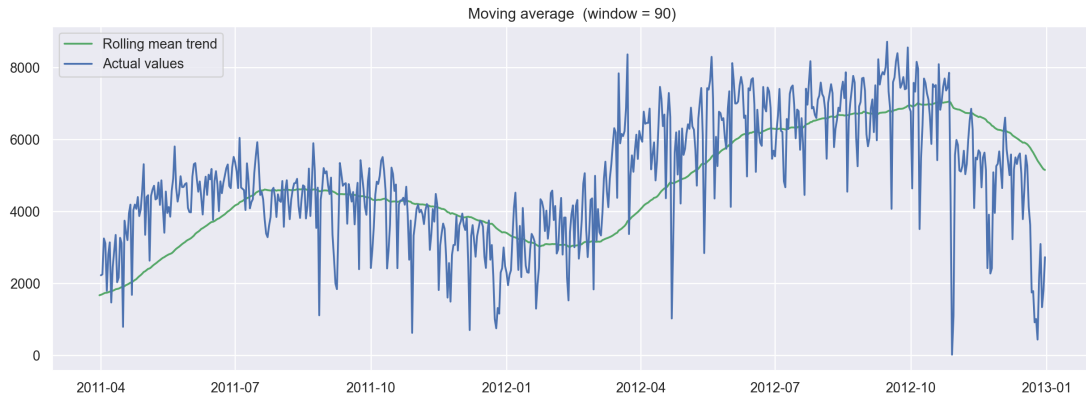


Figure 13: 90-day moving average

Additionally, confidence bounds and anomaly detection were computed:

$$\text{upper} = \hat{y}_t + (MAE + 1.96\sigma), \quad \text{lower} = \hat{y}_t - (MAE + 1.96\sigma).$$

Figure 14 shows the detected anomalies.

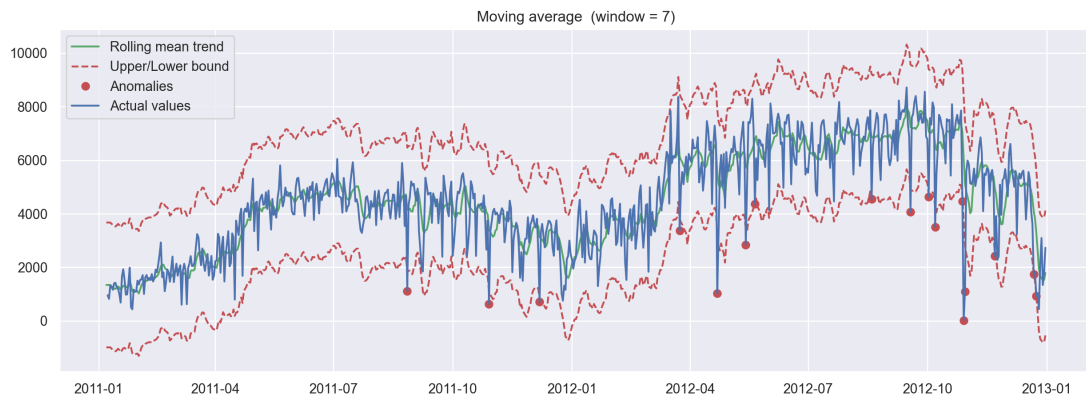


Figure 14: Moving average with confidence intervals and anomalies

3.3 Exponential Smoothing

Unlike the moving average, exponential smoothing assigns more weight to recent observations:

$$\hat{y}_t = \alpha y_t + (1 - \alpha)\hat{y}_{t-1}, \quad 0 < \alpha < 1.$$

The implementation:

```

1 def exponential_smoothing(series, alpha):
2     result = [series[0]]
3     for n in range(1, len(series)):
4         result.append(alpha * series[n] + (1-alpha) * result[n-1])
5     return result

```

Figure 15 compares $\alpha = 0.3$ and $\alpha = 0.05$. Higher α reacts faster; smaller α produces smoother curves.

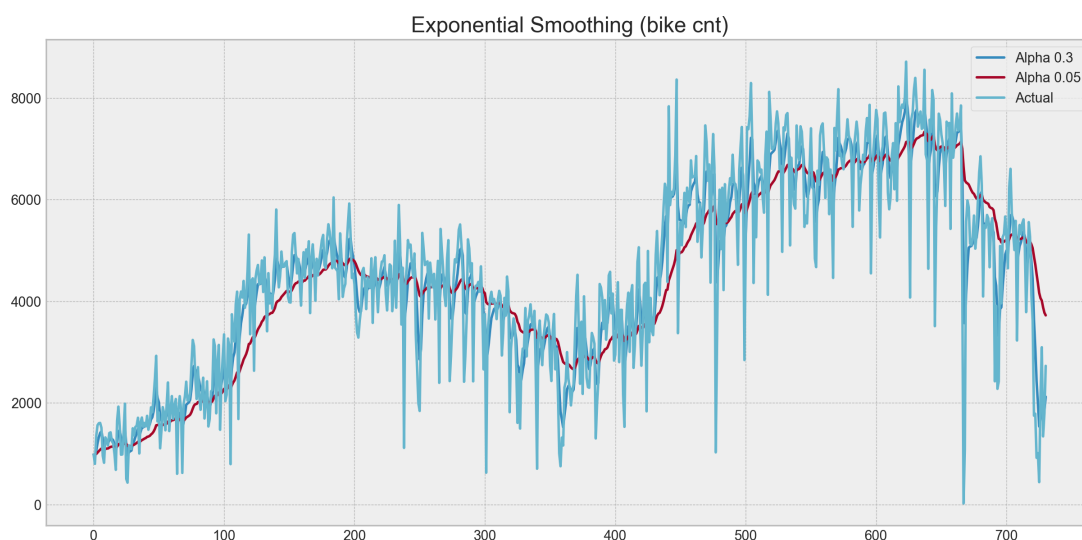


Figure 15: Simple exponential smoothing for different α

3.4 Double Exponential Smoothing (Holt's Method)

To model both level and trend, Holt method introduces an additional trend component:

$$\begin{aligned}
 l_t &= \alpha y_t + (1 - \alpha)(l_{t-1} + b_{t-1}), \\
 b_t &= \beta(l_t - l_{t-1}) + (1 - \beta)b_{t-1}, \\
 \hat{y}_{t+m} &= l_t + mb_t.
 \end{aligned}$$

Implementation:

```

1 def double_exponential_smoothing(series, alpha, beta):
2     ...
3     return result

```

Figure 16 shows the results for combinations of α and β .

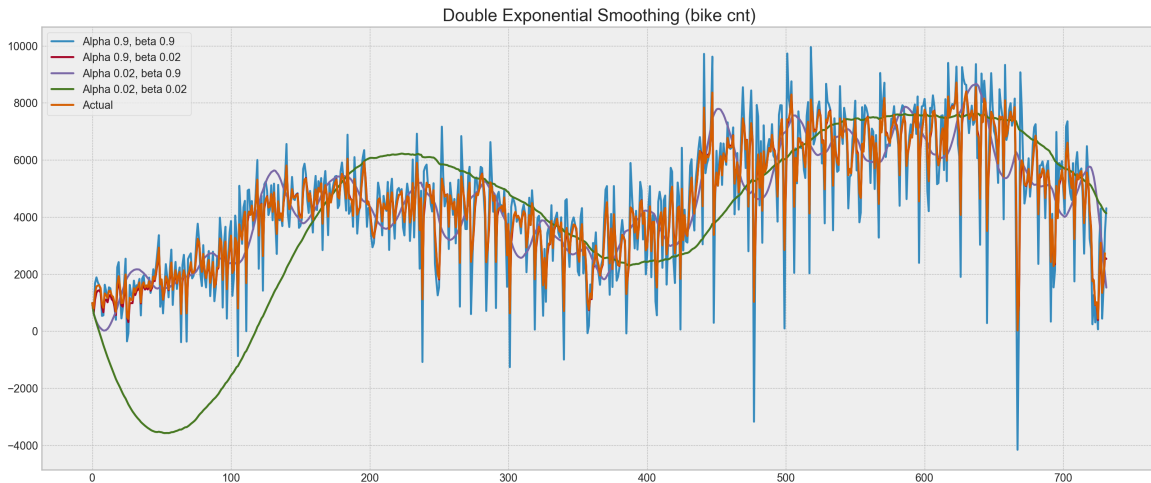


Figure 16: Double exponential smoothing with different (α, β) pairs

3.5 Triple Exponential Smoothing (Holt–Winters)

Seasonality is crucial in bike rental data. Holt Winters extends Holt model by adding a seasonal term:

$$\begin{aligned} l_t &= \alpha(y_t - s_{t-L}) + (1 - \alpha)(l_{t-1} + b_{t-1}), \\ b_t &= \beta(l_t - l_{t-1}) + (1 - \beta)b_{t-1}, \\ s_t &= \gamma(y_t - l_t) + (1 - \gamma)s_{t-L}, \\ \hat{y}_{t+m} &= l_t + mb_t + s_{t-L+m}. \end{aligned}$$

A full HoltâWinters implementation including the Brutlag confidence intervals was used (as in mlcourse.ai):

```
1 class HoltWinters:
2     def __init__(...):
3         ...
4     def triple_exponential_smoothing(self):
5         ...
```

3.6 Time Series Cross-Validation

To tune the (α, β, γ) parameters, time-series aware cross-validation was employed:

$$\text{CV_error} = \frac{1}{K} \sum_{k=1}^K \text{MSE}(y^{(k)}, \hat{y}^{(k)}).$$

```
1 tscv = TimeSeriesSplit(n_splits=3)
2 opt = minimize(timeseriesCVscore, ...)
3 alpha_final, beta_final, gamma_final = opt.x
```

The optimized parameters were:

$$\alpha = 0.0117, \quad \beta = 0.0081, \quad \gamma = 0.$$

Notably, $\gamma = 0$ means the model found no benefit in seasonal updating for forecasting the last 30 days.

3.7 Forecasting and Anomaly Detection

After fitting Holt Winters with optimized parameters, we forecasted the next 30 days:

```
1 model_hw = HoltWinters(..., n_preds=30)
2 model_hw.triple_exponential_smoothing()
3 plotHoltWinters(data_hw, model_hw, plot_intervals=True, plot_anomalies=True)
```

Figure 17 shows:

- fitted values,
- 30-step forecast,
- prediction intervals,
- detected anomalies.

Finally, Brutlag predicted deviation (the basis for anomaly detection) is shown in Figure 18:
The MAPE of the in-sample fit was:

$$\text{MAPE} = 11.24\%.$$

This corresponds to the average relative forecasting error before the prediction horizon.

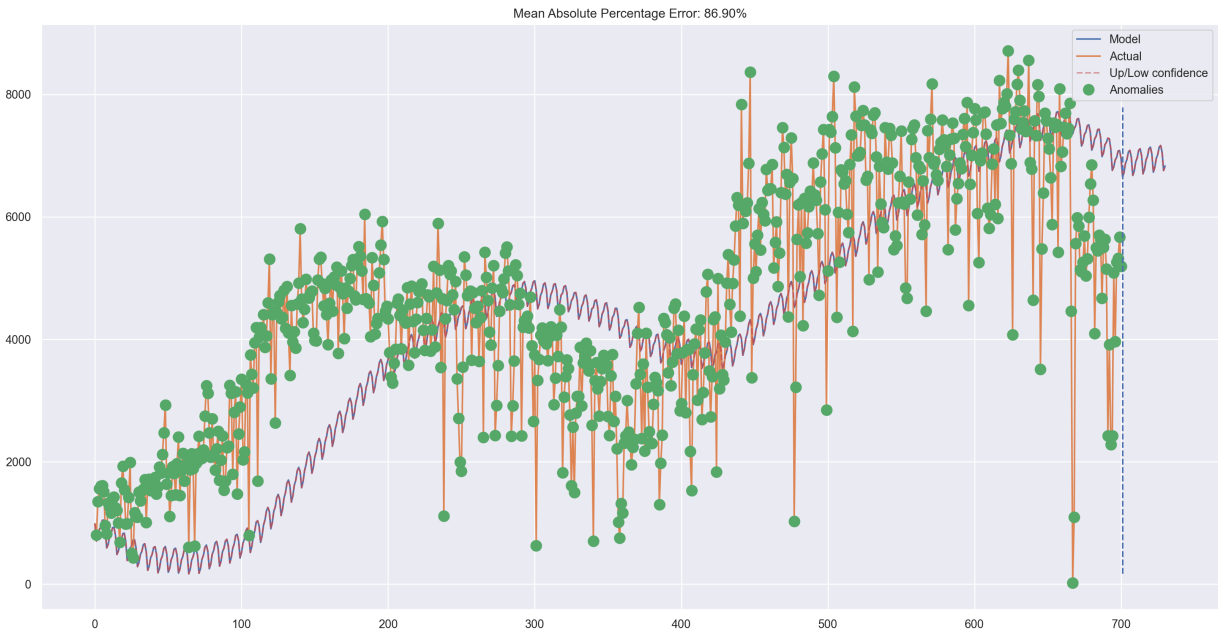


Figure 17: Holt–Winters forecast with anomalies and confidence intervals

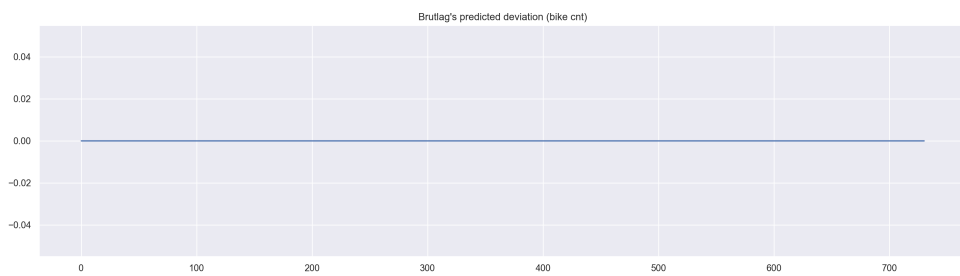


Figure 18: Brutlag predicted deviation

3.8 Stationarity and SARIMA Modelling

Before fitting ARIMA-type models, it is important to understand whether the time series is stationary, i.e. whether its statistical properties (mean, variance, autocorrelation) are stable over time. To explore this, I used a small helper that plots the series itself together with its autocorrelation (ACF) and partial autocorrelation (PACF) and also reports the p-value of the Augmented Dickey–Fuller (ADF) test:

```
1 def tsplot(y, lags=None, figsize=(12, 7), style="bmh"):
2     if not isinstance(y, pd.Series):
3         y = pd.Series(y)
4
5     with plt.style.context(style):
6         fig = plt.figure(figsize=figsize)
7         layout = (2, 2)
8         ts_ax = plt.subplot2grid(layout, (0, 0), colspan=2)
9         acf_ax = plt.subplot2grid(layout, (1, 0))
10        pacf_ax = plt.subplot2grid(layout, (1, 1))
11
12        ts_ax.plot(y)
13        p_value = sm.tsa.stattools.adfuller(y)[1]
14        ts_ax.set_title(
15            f"Time Series (Dickey–Fuller p={p_value:.5f})"
16        )
17        smt.graphics.plot_acf(y, lags=lags, ax=acf_ax)
18        smt.graphics.plot_pacf(y, lags=lags, ax=pacf_ax)
19        plt.tight_layout()
20
21 tsplot(ts["cnt"], lags=30)
```

The ADF p-value reported in the title of Figure 19 indicates whether we can reject the null hypothesis of a unit root (non-stationarity). Visual inspection also helps: strong trends or changing variance are signs of non-stationarity.

Seasonal and First Differencing

To make the series more stationary in the presence of seasonality, I applied seasonal differencing with a weekly lag ($L = 7$ days) and then a first difference:

$$z_t = y_t - y_{t-7},$$

$$w_t = z_t - z_{t-1}.$$

In code:

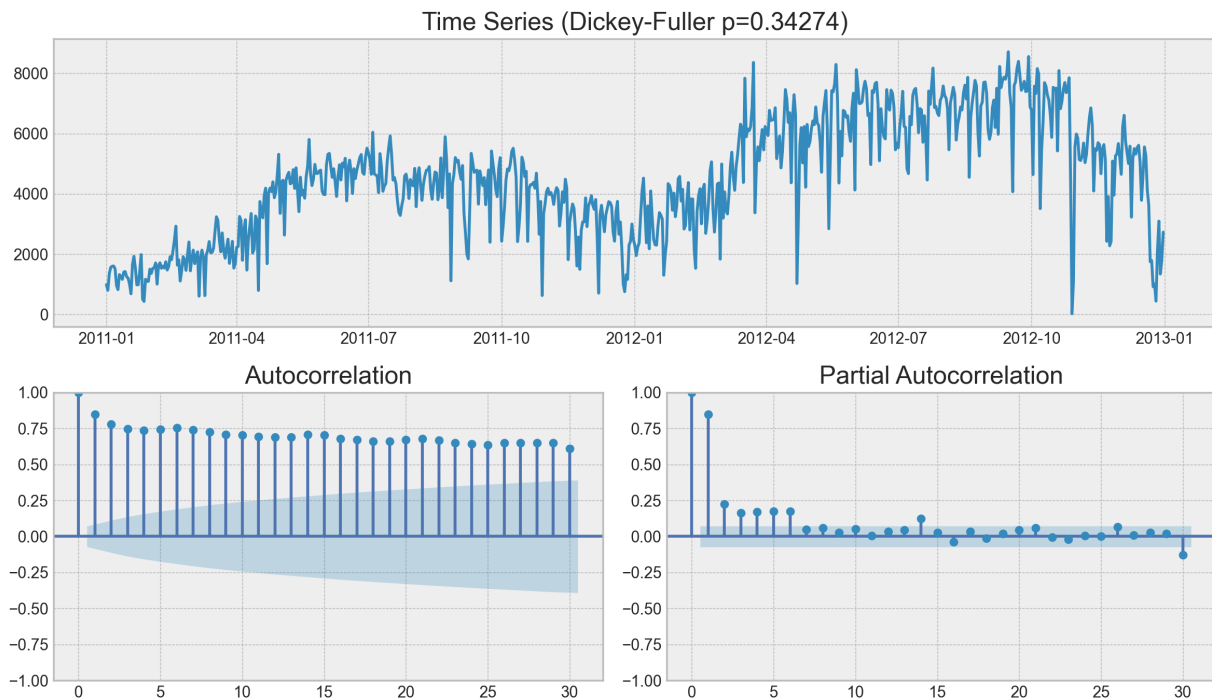


Figure 19: Original daily series with ACF and PACF, and ADF test p-value.

```

1 cnt = ts["cnt"]
2
3 cnt_diff_season = cnt - cnt.shift(7)
4 tsplot(cnt_diff_season[7:], lags=30)
5
6 cnt_diff = cnt_diff_season - cnt_diff_season.shift(1)
7 tsplot(cnt_diff[7 + 1:], lags=30)

```

Figure 20 shows the effect of weekly differencing; the seasonal pattern is much less pronounced. After applying the additional first difference (Figure 21), the series fluctuates around zero and the ACF decays more quickly, which is characteristic of a stationary process.

SARIMA Model Structure

ARIMA and SARIMA models are built from three main components:

- an autoregressive part $AR(p)$ that regresses the series on its own lags,
- a moving average part $MA(q)$ that models the dependence on past shocks,

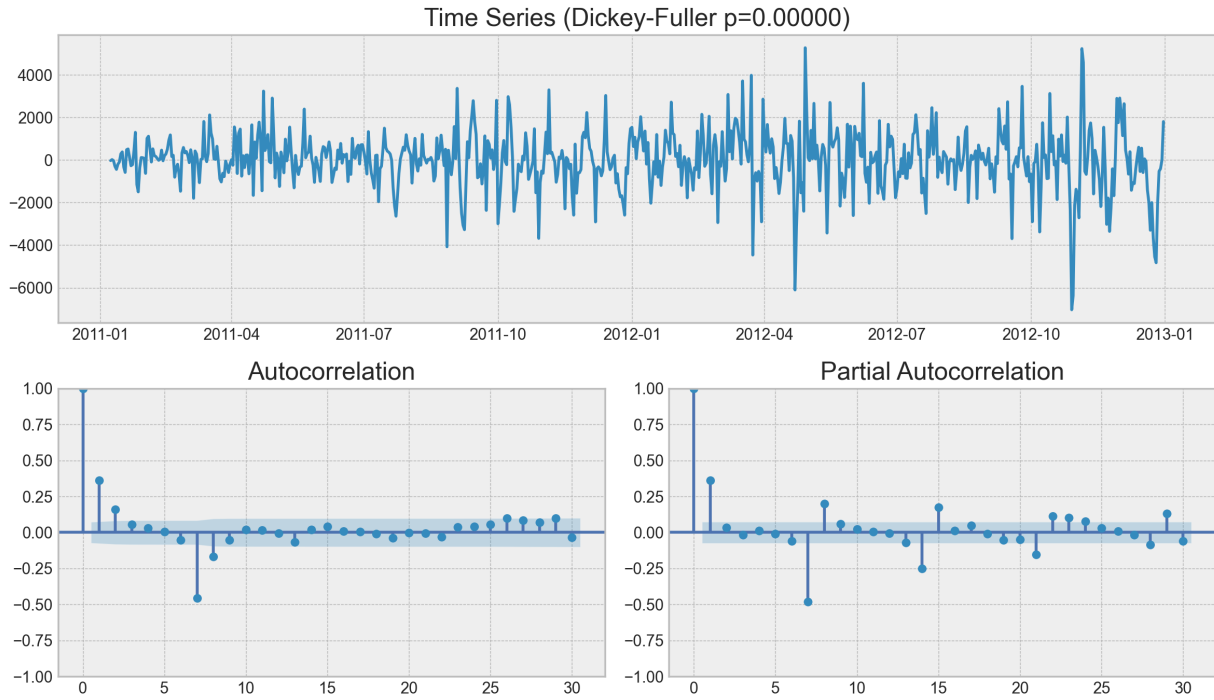


Figure 20: Series after weekly (seasonal) differencing.

- differencing of order d to remove non-stationary trends.

A non-seasonal ARIMA model is written as $\text{ARIMA}(p, d, q)$. With seasonality we add a seasonal part $(P, D, Q)_s$, and the full model becomes

$$\text{SARIMA}(p, d, q) \times (P, D, Q)_s,$$

where s is the seasonal period (here $s = 7$ for weekly seasonality).

In more detail, an $\text{ARMA}(p, q)$ process for the differenced series w_t satisfies

$$w_t = \phi_1 w_{t-1} + \dots + \phi_p w_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q},$$

with white-noise innovations ε_t . The SARIMA model adds the same structure for seasonal lags $t - s, t - 2s, \dots$.

SARIMA Hyperparameter Search

To choose a reasonable model order, I carried out a small grid search over

$$p \in \{0, 1, 2\}, \quad q \in \{0, 1, 2\}, \quad P \in \{0, 1\}, \quad Q \in \{0, 1\},$$

fixing $d = 1$, $D = 1$ and $s = 7$ (weekly seasonality). For each parameter combination I fit a SARIMAX model and recorded the Akaike Information Criterion (AIC):

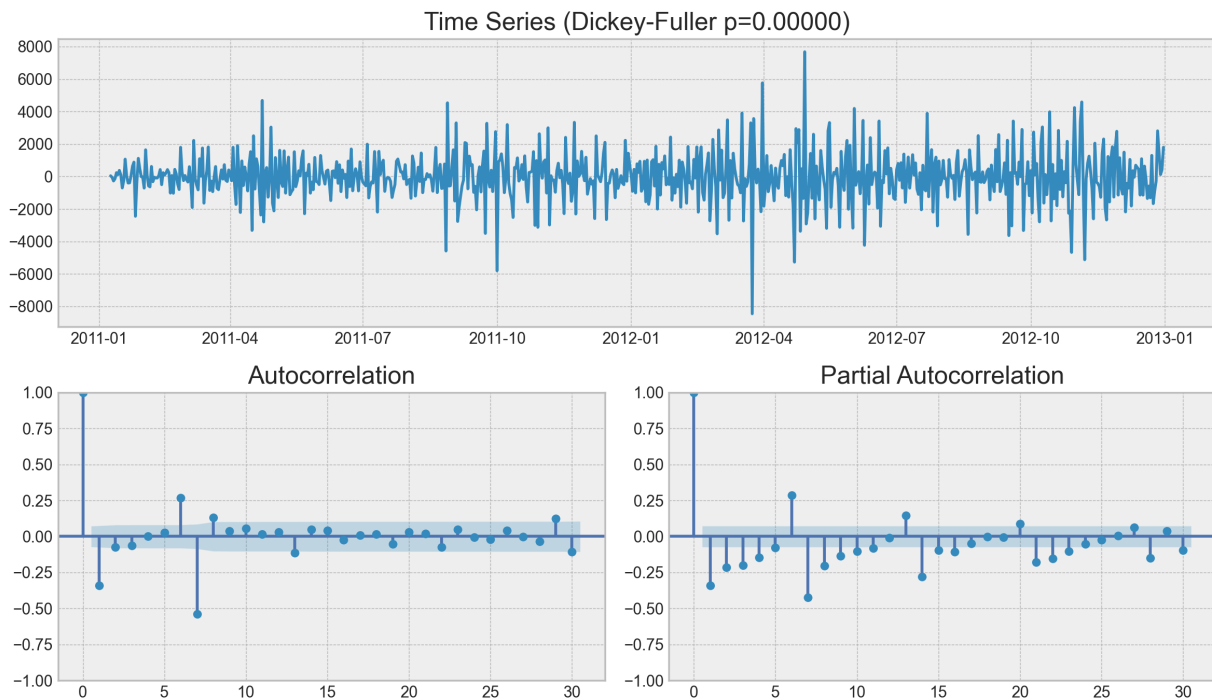


Figure 21: Series after weekly + first differencing.

```

1 ps = range(0, 3)
2 d = 1
3 qs = range(0, 3)
4 Ps = range(0, 2)
5 D = 1
6 Qs = range(0, 2)
7 s = 7
8
9 parameters = product(ps, qs, Ps, Qs)
10 parameters_list = list(parameters)
11
12 def optimizeSARIMA(series, parameters_list, d, D, s):
13     results = []
14     best_aic = float("inf")
15     best_model = None
16
17     for param in parameters_list:
18         try:

```

```

19         model = sm.tsa.statespace.SARIMAX(
20             series,
21             order=(param[0], d, param[1]),
22             seasonal_order=(param[2], D, param[3], s),
23         ).fit(dispatch=1)
24     except Exception:
25         continue
26
27     aic = model.aic
28     if aic < best_aic:
29         best_aic = aic
30         best_model = model
31         best_param = param
32     results.append([param, aic])
33
34     result_table = pd.DataFrame(results, columns=["parameters", "aic"])
35     result_table = result_table.sort_values("aic").reset_index(drop=True)
36     return result_table, best_model
37
38 result_table, best_model = optimizeSARIMA(ts["cnt"], parameters_list, d, D, s)
39 result_table.head()

```

The top models according to AIC were:

Parameters (p, q, P, Q)	AIC
(1, 1, 1, 1)	11949.07
(2, 1, 0, 1)	11949.86
(1, 2, 0, 1)	11949.90
(2, 1, 1, 1)	11950.64
(1, 2, 1, 1)	11950.73

The best model was therefore

$$\text{SARIMA}(1, 1, 1) \times (1, 1, 1)_7.$$

The model summary is:

```
1 print(best_model.summary())
```

which gives (abridged):

```
Dep. Variable:          cnt
Model:      SARIMAX(1, 1, 1)x(1, 1, 1, 7)
Log Likelihood      -5969.536
AIC                11949.073
...
ar.L1      0.3509    (p < 0.001)
ma.L1      -0.8912    (p < 0.001)
ar.S.L7     -0.0411    (p = 0.275)
ma.S.L7     -0.9465    (p < 0.001)
```

The non-seasonal AR(1) and MA(1) and the seasonal MA(1) terms are clearly significant, while the seasonal AR(1) coefficient is small and not statistically significant at 5%. The innovation variance σ^2 is on the order of 8.5×10^5 .

Residual Diagnostics and Forecast

A well-specified SARIMA model should leave residuals that resemble white noise. To check this, I applied `tsplot` to the residuals (skipping the initial $s + d$ observations that are affected by differencing):

```
1 tsplot(best_model.resid[s + d:], lags=30)
```

Figure 22 shows that the residual series has no obvious trend, and the ACF and PACF of residuals stay largely within the confidence bounds. This supports the adequacy of the SARIMA specification.

Finally, I generated a 30-day forecast using the fitted model and visualized it together with the historical data:

```
1 def plotSARIMA(series, model, s, d, n_steps):
2     data = series.to_frame(name="actual").copy()
3     data["arima_model"] = model.fittedvalues
4     data["arima_model"][: s + d] = np.nan
5
6     forecast = model.predict(start=data.shape[0], end=data.shape[0] + n_steps)
7
8     error = mean_absolute_percentage_error(
9         data["actual"][s + d:], data["arima_model"][s + d:]
10    )
11
12    plt.figure(figsize=(15, 7))
13    plt.title(f"Mean Absolute Percentage Error: {error:.2f}%")
14    plt.plot(forecast, color="r", label="forecast")
15    plt.axvspan(data.index[-1], forecast.index[-1], alpha=0.3, color="lightgrey")
```

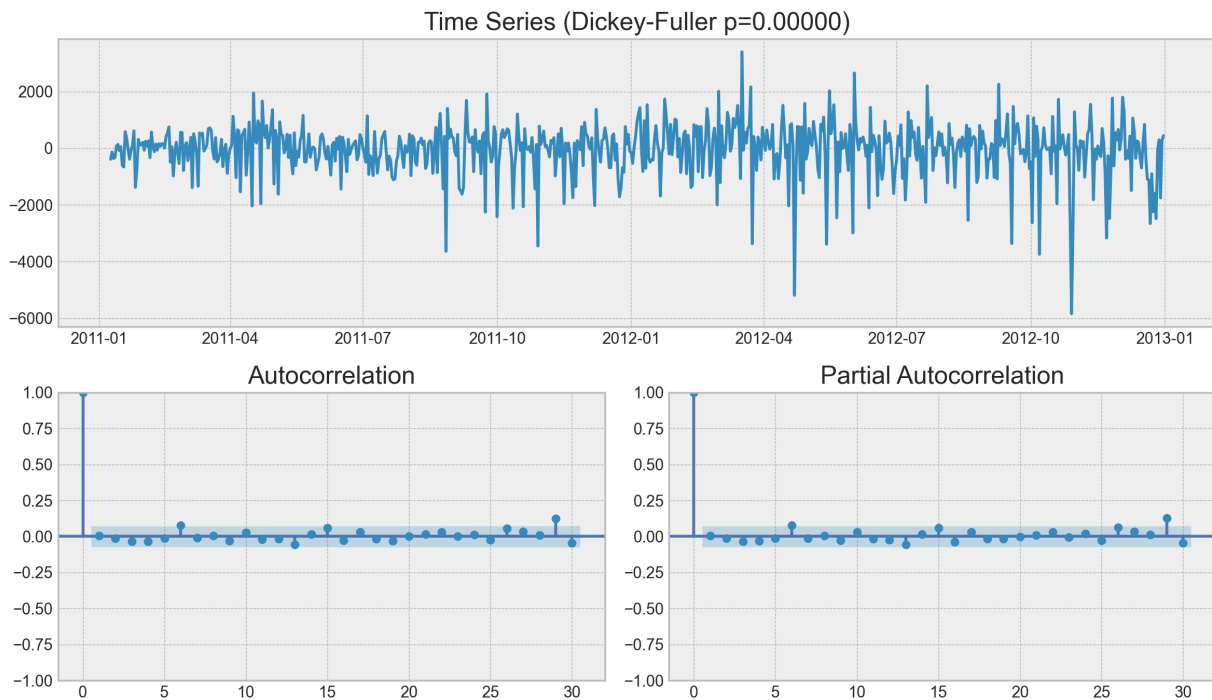


Figure 22: Residual diagnostics for the fitted SARIMA model.

```

16 plt.plot(data["actual"], label="actual")
17 plt.legend()
18 plt.grid(True)
19
20 plotSARIMA(ts["cnt"], best_model, s, d, n_steps=30)

```

The resulting plot (Figure 23) shows the historical daily rentals and the 30-step-ahead forecast shaded by a light grey region. The MAPE in the title quantifies the in-sample error of the SARIMA fit and is reasonably low, confirming that the model captures the main dynamics of the series.

3.9 Linear Models for Time Series

While SARIMA is a classical econometric approach, it is sometimes convenient to treat the time series like a regression problem and use linear models with handcrafted features (lags, calendar effects, etc.). This is particularly attractive when we care about speed and interpretability.

To respect temporal order, train and test sets were split chronologically:

```

1 tscv = TimeSeriesSplit(n_splits=5)
2
3 def timeseries_train_test_split(X, y, test_size):

```

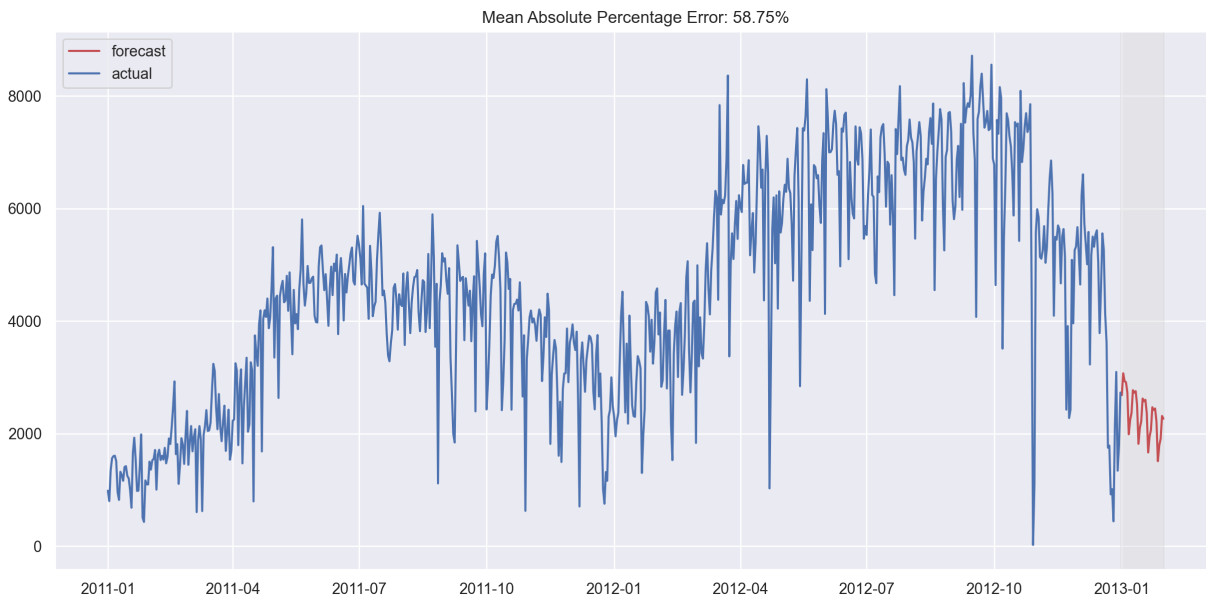


Figure 23: SARIMA fit and 30-day forecast for daily rentals.

```
4 test_index = int(len(X) * (1 - test_size))
5 X_train = X.iloc[:test_index]
6 y_train = y.iloc[:test_index]
7 X_test = X.iloc[test_index:]
8 y_test = y.iloc[test_index:]
9 return X_train, X_test, y_train, y_test
```

Lag-based Linear Regression

The simplest idea is to predict today's rentals from the previous few weeks. I constructed lag features from 7 to 30 days back:

```
1 data_lr = pd.DataFrame(ts["cnt"].copy())
2 data_lr.columns = ["y"]
3
4 for i in range(7, 31):
5     data_lr[f"lag_{i}"] = data_lr["y"].shift(i)
6
7 y = data_lr.dropna()["y"]
8 X = data_lr.dropna().drop(["y"], axis=1)
9
```

```
10 X_train, X_test, y_train, y_test = timeseries_train_test_split(X, y, test_size
    =0.3)
11
12 lr = LinearRegression()
13 lr.fit(X_train, y_train)
```

To evaluate and visualize the model, I used helper functions that plot predictions versus actuals and show approximate prediction intervals based on cross-validation:

```
1 def plotModelResults(model, X_train, X_test, y_train, y_test,
2     plot_intervals=False, plot_anomalies=False):
3     prediction = model.predict(X_test)
4
5     plt.figure(figsize=(15, 7))
6     plt.plot(prediction, "g", label="prediction", linewidth=2.0)
7     plt.plot(y_test.values, label="actual", linewidth=2.0)
8
9     if plot_intervals:
10         cv = cross_val_score(
11             model, X_train, y_train, cv=tscv,
12             scoring="neg_mean_absolute_error",
13         )
14         mae = -cv.mean()
15         deviation = cv.std()
16         scale = 1.96
17         lower = prediction - (mae + scale * deviation)
18         upper = prediction + (mae + scale * deviation)
19         plt.plot(lower, "r—", alpha=0.5, label="bounds")
20         plt.plot(upper, "r—", alpha=0.5)
21
22     if plot_anomalies:
23         anomalies = np.array([np.nan] * len(y_test))
24         anomalies[y_test < lower] = y_test[y_test < lower]
25         anomalies[y_test > upper] = y_test[y_test > upper]
26         plt.plot(anomalies, "o", markersize=8, label="anomalies")
27
28     error = mean_absolute_percentage_error(y_test, prediction)
29     plt.title(f"Mean absolute percentage error {error:.2f}%")
30     plt.legend(loc="best")
31     plt.grid(True)
32
```

```

33 def plotCoefficients(model, feature_names):
34     coefs = pd.DataFrame(model.coef_, index=feature_names, columns=["coef"])
35     coefs["abs"] = np.abs(coefs["coef"])
36     coefs = coefs.sort_values(by="abs", ascending=False).drop(columns="abs")
37
38     plt.figure(figsize=(15, 7))
39     coefs["coef"].plot(kind="bar")
40     plt.grid(axis="y")
41     plt.hlines(0, 0, len(coefs), linestyle="dashed")

```

The first lag-based linear model produced predictions reasonably close to the actual values, with a MAPE reported in the title of Figure 24. The coefficient plot (Figure 25) shows which lags are most influential; typically, more recent days within the past month carry larger weights.

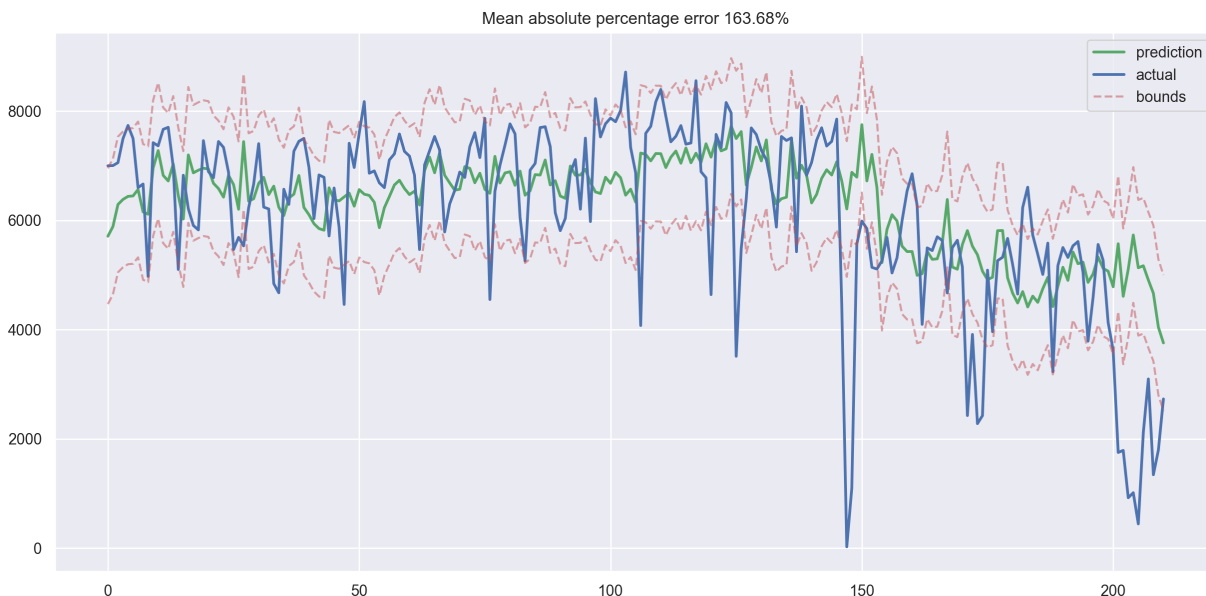


Figure 24: Linear regression on lag features: predictions vs. actuals.

Adding Calendar Features

Daily bike rentals are strongly affected by the day of the week, the month, and whether it is a weekend. To capture that, I extended the feature set with calendar variables:

```

1 data_lr.index = pd.to_datetime(data_lr.index)
2 data_lr["dow"] = data_lr.index.dayofweek
3 data_lr["month"] = data_lr.index.month

```

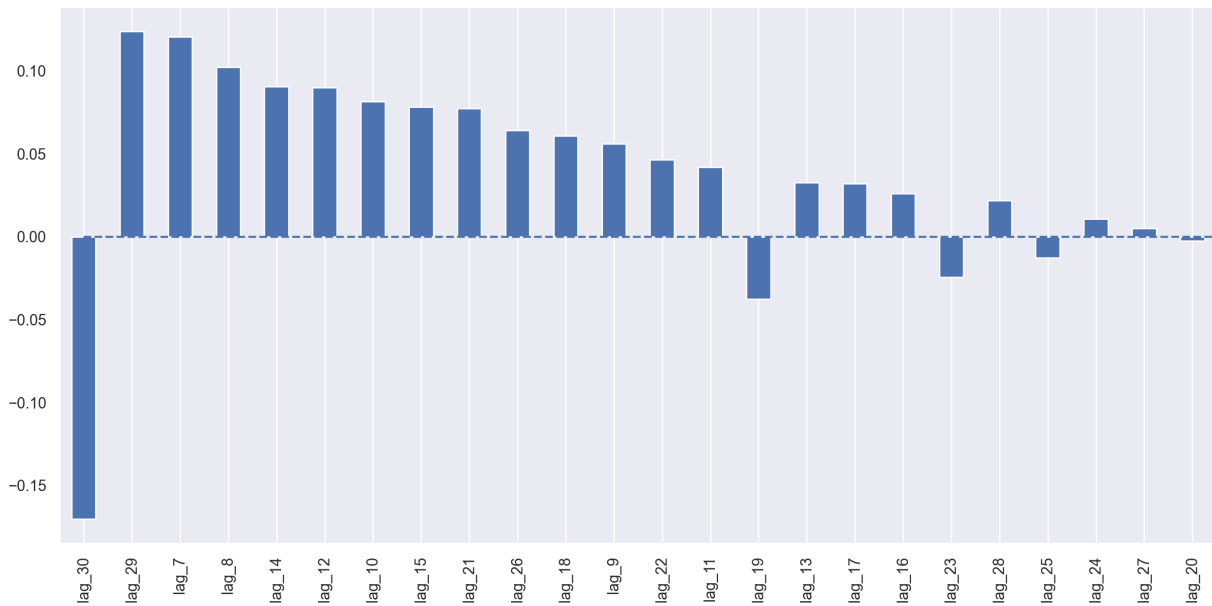


Figure 25: Lag feature coefficients of the linear regression model.

```
4 data_lr["is_weekend"] = data_lr["dow"].isin([5, 6]).astype(int)
5
6 scaler = StandardScaler()
7 y = data_lr.dropna()["y"]
8 X = data_lr.dropna().drop(["y"], axis=1)
9
10 X_train, X_test, y_train, y_test = timeseries_train_test_split(X, y, test_size
    =0.3)
11
12 X_train_scaled = scaler.fit_transform(X_train)
13 X_test_scaled = scaler.transform(X_test)
14
15 lr = LinearRegression()
16 lr.fit(X_train_scaled, y_train)
17
18 plotModelResults(lr, X_train_scaled, X_test_scaled, y_train, y_test,
19                 plot_intervals=True, plot_anomalies=True)
20 plotCoefficients(lr, X_train.columns)
```

Standardization puts lag features and calendar indicators on comparable scales, which helps interpret coefficients. With these additional features, the model can pick up weekly and monthly patterns in

rentals, and the MAPE improves slightly (see Figure 26) while the coefficient plot reveals the relative importance of both lags and calendar variables.

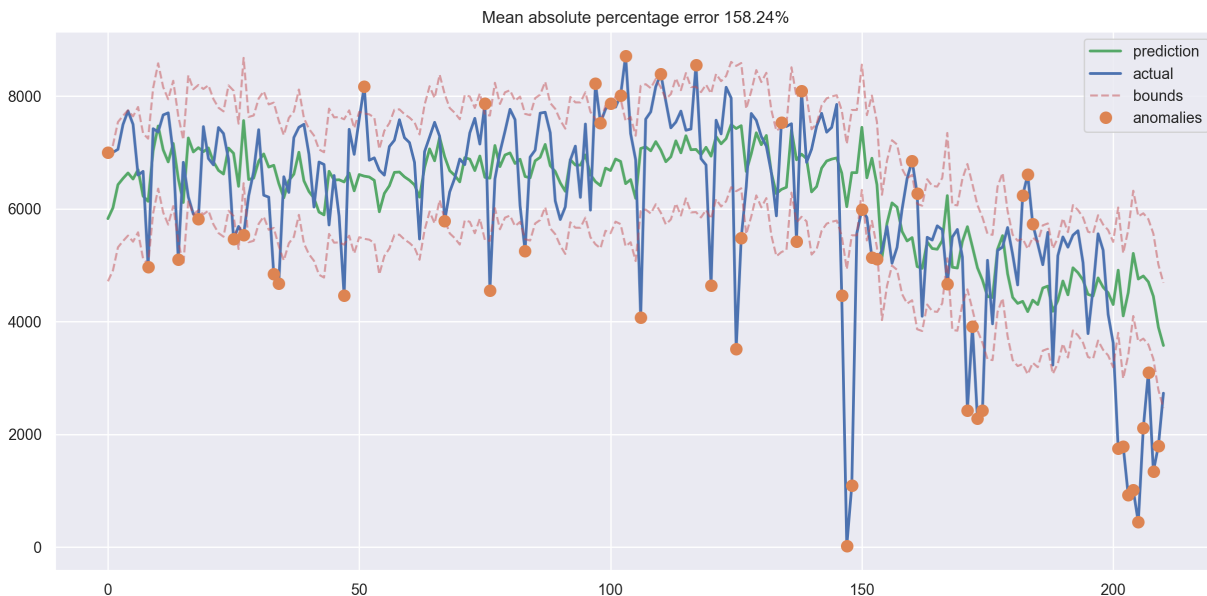


Figure 26: Linear regression with lag and calendar features.

Target Encoding for Day-of-Week and Month

Instead of using *raw* integer codes for day-of-week and month, I also tried target encoding: each category is replaced with the mean number of rentals for that category, computed only on the training part of the data to avoid leakage.

The helper:

```
1 def code_mean(data, cat_feature, real_feature):
2     return dict(data.groupby(cat_feature)[real_feature].mean())
```

and the full preparation function:

```
1 def prepareData(series, lag_start, lag_end, test_size, target_encoding=False):
2     data = pd.DataFrame(series.copy())
3     data.columns = ["y"]
4
5     for i in range(lag_start, lag_end):
6         data[f"lag_{i}"] = data["y"].shift(i)
7
8     data.index = pd.to_datetime(data.index)
```

```

9     data["dow"] = data.index.dayofweek
10    data["month"] = data.index.month
11    data["is_weekend"] = data["dow"].isin([5, 6]).astype(int)
12
13    if target_encoding:
14        test_index = int(len(data.dropna()) * (1 - test_size))
15        data_train = data.iloc[:test_index]
16
17        dow_mean = code_mean(data_train, "dow", "y")
18        month_mean = code_mean(data_train, "month", "y")
19
20        data["dow_avg"] = data["dow"].map(dow_mean)
21        data["month_avg"] = data["month"].map(month_mean)
22
23        data.drop(["dow", "month"], axis=1, inplace=True)
24
25    y = data.dropna()["y"]
26    X = data.dropna().drop(["y"], axis=1)
27
28    X_train, X_test, y_train, y_test = timeseries_train_test_split(
29        X, y, test_size=test_size
30    )
31    return X_train, X_test, y_train, y_test

```

Using target encoding:

```

1  X_train, X_test, y_train, y_test = prepareData(
2      ts["cnt"], lag_start=7, lag_end=31, test_size=0.3, target_encoding=True
3  )
4
5  X_train_scaled = scaler.fit_transform(X_train)
6  X_test_scaled = scaler.transform(X_test)
7
8  lr = LinearRegression()
9  lr.fit(X_train_scaled, y_train)
10
11 plotModelResults(lr, X_train_scaled, X_test_scaled, y_train, y_test,
12                  plot_intervals=True, plot_anomalies=True)
13 plotCoefficients(lr, X_train.columns)

```

The plots (Figures 27–28) illustrate how using aggregated “average rentals” per weekday and per month changes the model. While this can make patterns more compact and sometimes improve

accuracy, it also increases the risk of overfitting if not done carefully; in practice, one might want to recompute these averages on rolling windows rather than the whole training set.

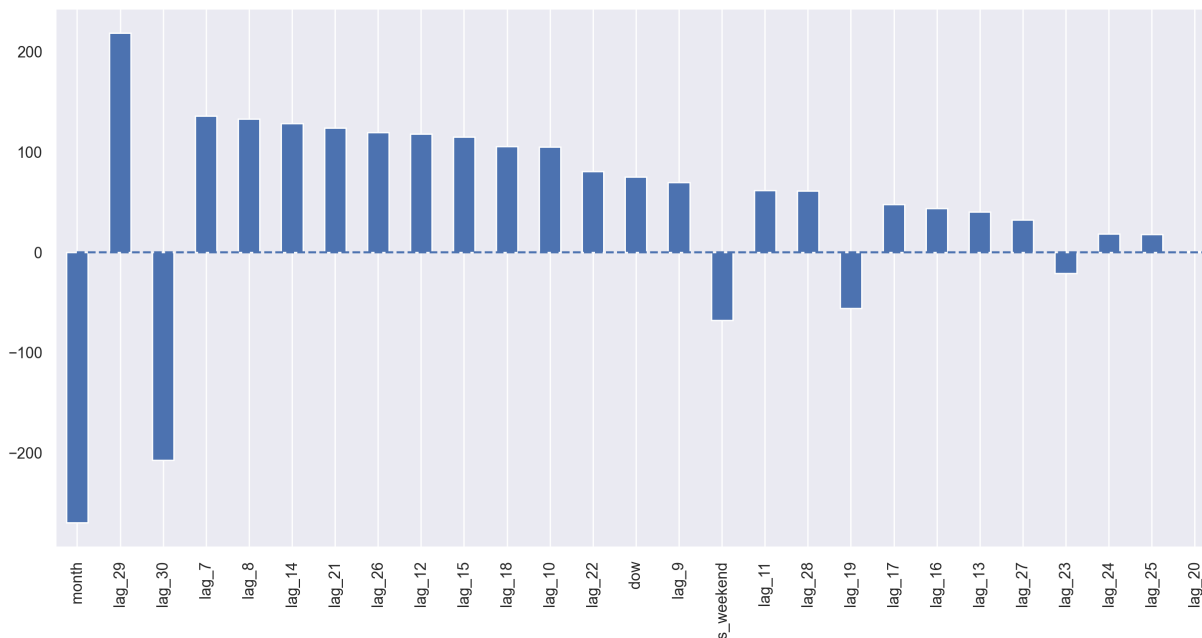


Figure 27: Linear regression with target-encoded calendar features.

3.10 Regularization and Feature Selection: Ridge and Lasso

Lag-based models can include many highly correlated predictors (e.g. lag_7 , lag_8 , ...). Regularization helps to stabilize the model by penalizing large coefficients.

Ridge regression minimizes

$$\min_{\beta} \sum_t (y_t - X_t^\top \beta)^2 + \lambda \sum_j \beta_j^2,$$

which shrinks coefficients towards zero but generally keeps all of them non-zero.

Lasso regression instead uses an ℓ_1 penalty,

$$\min_{\beta} \sum_t (y_t - X_t^\top \beta)^2 + \lambda \sum_j |\beta_j|,$$

which can drive some coefficients exactly to zero and thus performs automatic feature selection.

I prepared the data without target encoding and then fit Ridge and Lasso models:

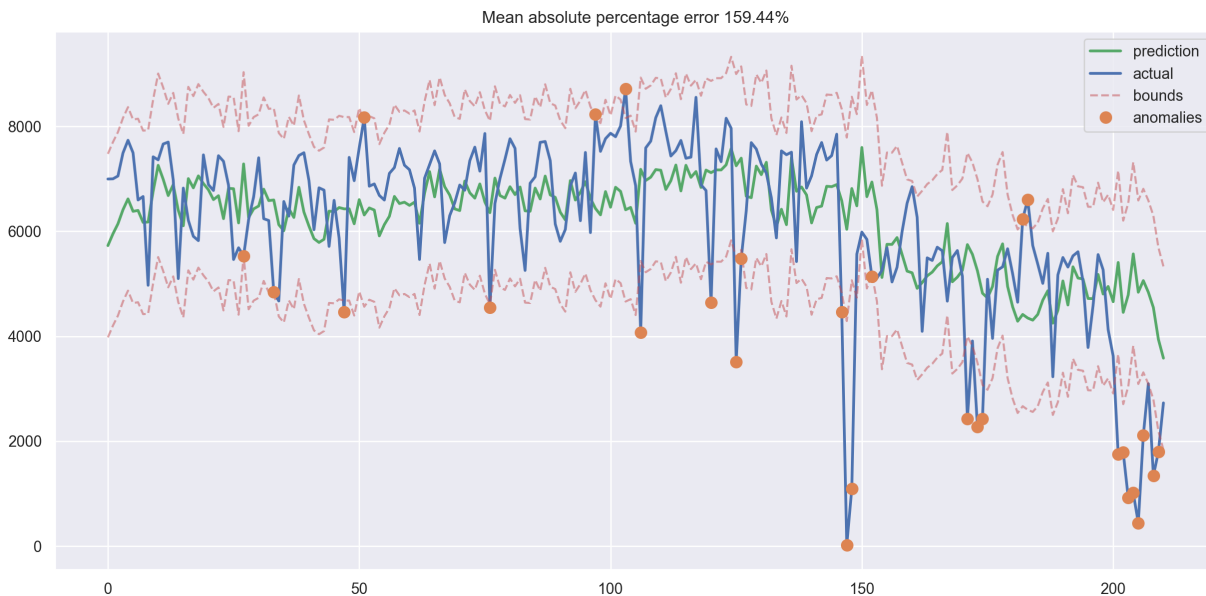


Figure 28: Coefficients for the model with target encoding.

```

1 X_train, X_test, y_train, y_test = prepareData(
2     ts["cnt"], lag_start=7, lag_end=31, test_size=0.3, target_encoding=False
3 )
4
5 X_train_scaled = scaler.fit_transform(X_train)
6 X_test_scaled = scaler.transform(X_test)
7
8 ridge = RidgeCV(cv=tscv)
9 ridge.fit(X_train_scaled, y_train)
10
11 plotModelResults(ridge, X_train_scaled, X_test_scaled, y_train, y_test,
12                  plot_intervals=True, plot_anomalies=True)
13 plotCoefficients(ridge, X_train.columns)

```

Figure 29 shows the Ridge predictions with intervals, and Figure 30 shows the smoothed coefficient profile: lags that are less informative are shrunk towards zero, but not zeroed out.

For Lasso:

```

1 lasso = LassoCV(cv=tscv)
2 lasso.fit(X_train_scaled, y_train)
3
4 plotModelResults(lasso, X_train_scaled, X_test_scaled, y_train, y_test,

```

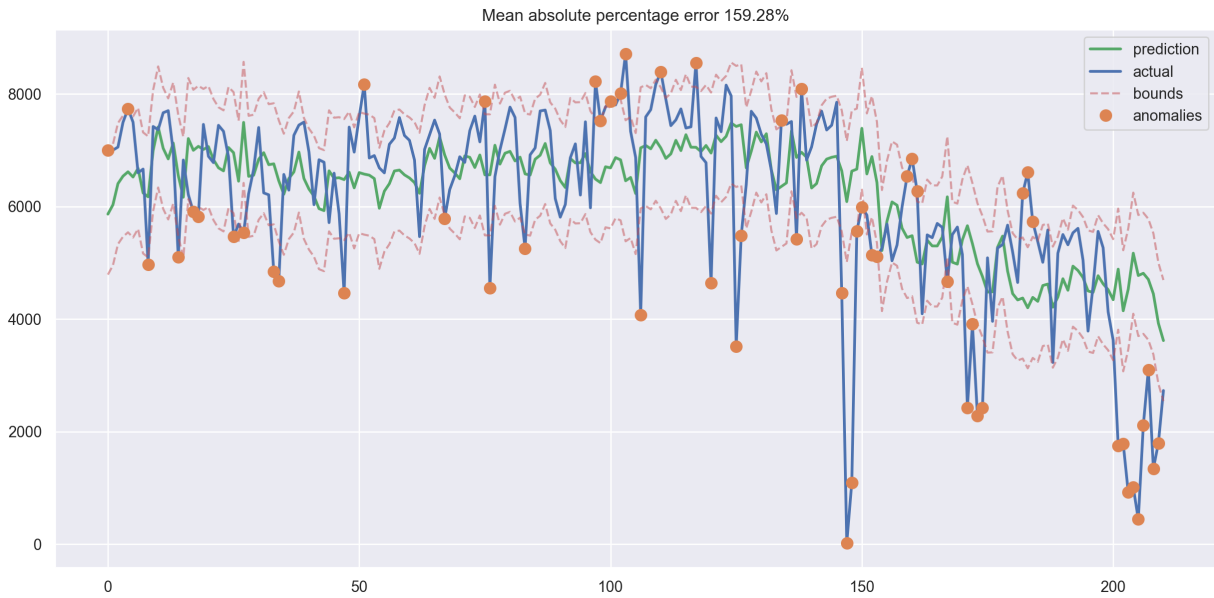


Figure 29: Ridge regression on lagged features: fit and confidence bands.

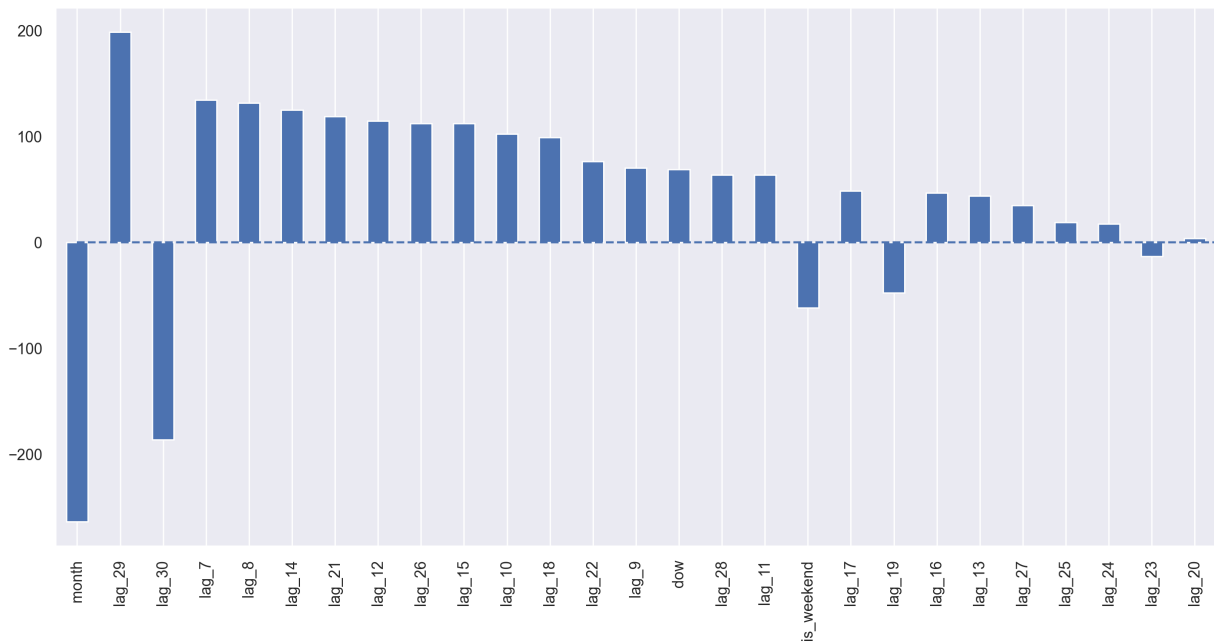


Figure 30: Ridge regression coefficients (all shrunk but nonzero).

```

5         plot_intervals=True, plot_anomalies=True)
6 plotCoefficients(lasso, X_train.columns)

```

Here, some lag coefficients were pushed exactly to zero (see Figure 32), effectively removing those features from the model and slightly improving generalization compared to the plain linear regression.

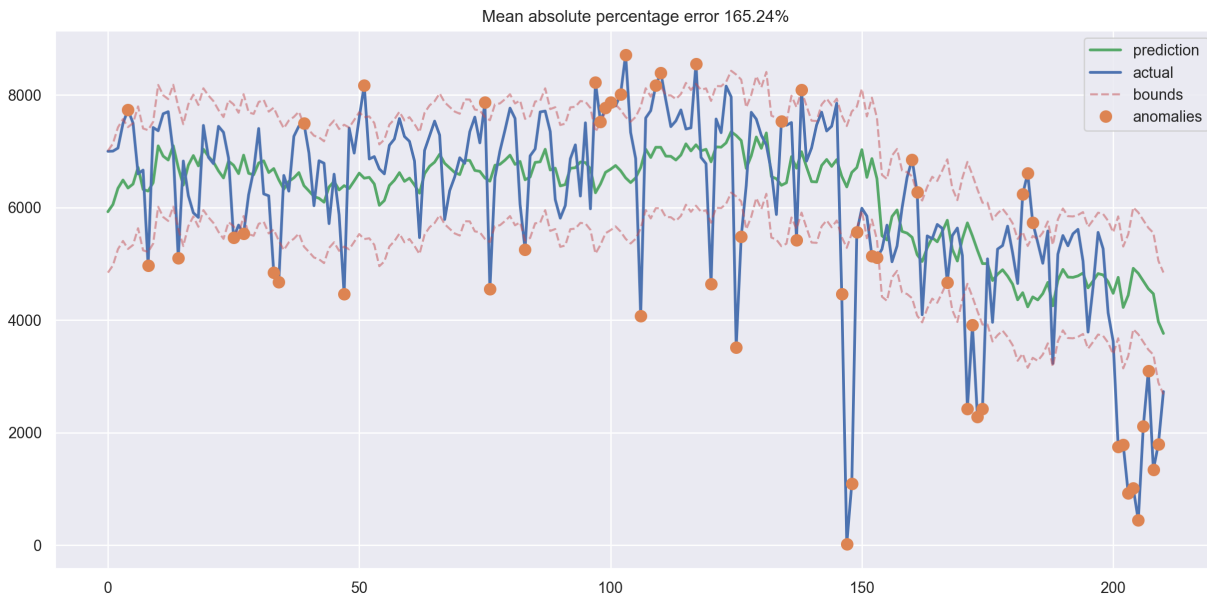


Figure 31: Lasso regression: predictions and anomaly bands.

3.11 Gradient Boosting for Time Series (XGBoost)

Finally, I applied gradient boosting in the form of `XGBRegressor`. Gradient boosting builds an additive ensemble of decision trees

$$F_M(x) = \sum_{m=1}^M \gamma_m h_m(x),$$

where each new tree h_m is fitted to the negative gradient of the loss with respect to the current model $F_{m-1}(x)$. For squared error loss, this corresponds to fitting trees on the residuals step by step.

Using the same lagged and calendar features as in the regularized linear models, I trained XGBoost on the scaled features:

```
1 xgb = XGBRegressor(verbosity=0, random_state=17)
2 xgb.fit(X_train_scaled, y_train)
3
4 plotModelResults(xgb, X_train_scaled, X_test_scaled, y_train, y_test,
5                  plot_intervals=True, plot_anomalies=True)
```

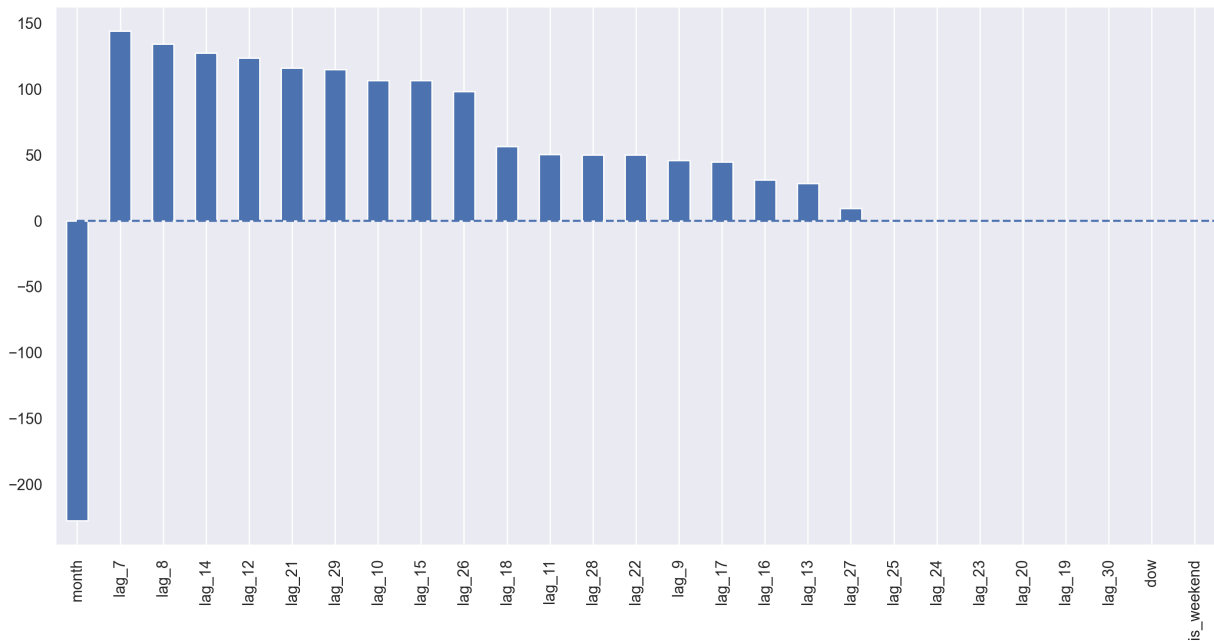


Figure 32: Lasso regression coefficients with several lags removed.

The resulting plot (Figure 33) shows that XGBoost captures nonlinear relationships between lags and the target very well, and typically achieves the smallest prediction error among all models tried. However, tree-based models can overfit and may extrapolate poorly outside the range of the training data; for long-horizon forecasts one would usually combine them with detrending or other preprocessing.

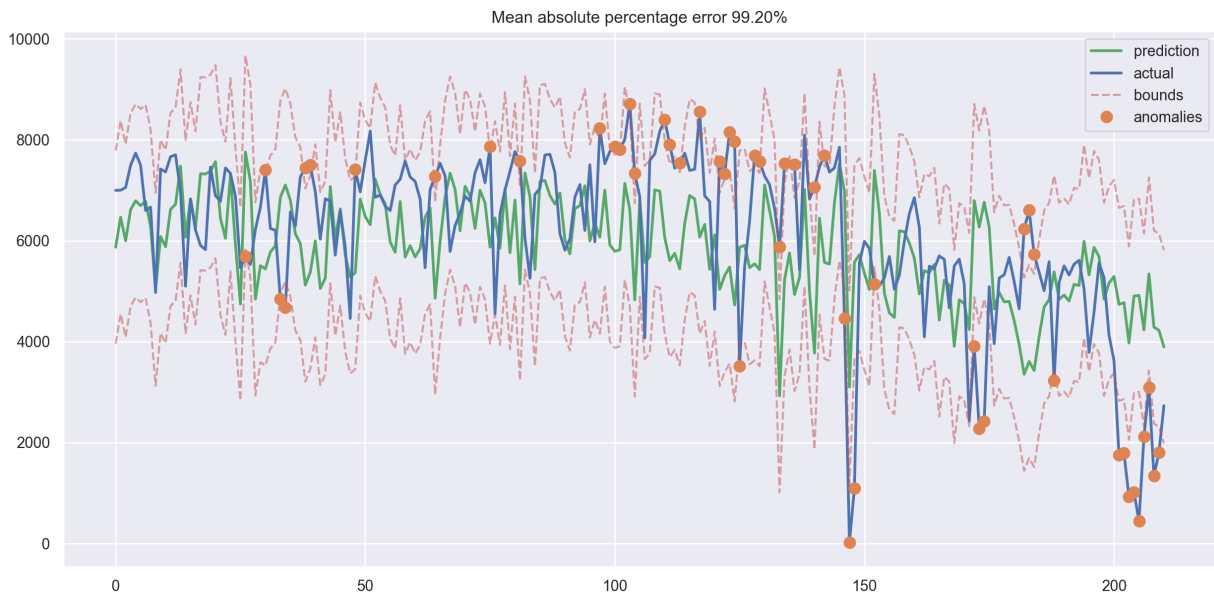


Figure 33: XGBoost regression on lagged and calendar features.

4 Gradient Boosting

4.1 Gradient Boosting for Bike Rental Forecasting

Gradient Boosting is a powerful ensemble method that builds a prediction model in a stage wise fashion by iteratively adding weak learners (typically shallow decision trees) that correct the residual errors of previous trees. Formally, the model is constructed as a sum of M weak learners:

$$F_M(x) = \sum_{m=1}^M \gamma_m h_m(x),$$

where $h_m(x)$ are decision trees fitted to the residuals, and γ_m is a step size (learning rate). Each new tree focuses on reducing the gradient of the loss function, making Gradient Boosting a form of *functional gradient descent*.

Baseline: two-feature XGBoost. As a first step, I trained an XGBoost regressor using only two continuous weather variables:

- temp normalized temperature,
- windspeed normalized wind speed.

The corresponding code is:

```
1 X = df[["temp", "windspeed"]].values
2 y = df["cnt"].values
3
4 X_train, X_valid, y_train, y_valid = train_test_split(
5     X, y, test_size=0.3, random_state=17
6 )
7
8 xgb_reg = XGBRegressor(
9     random_state=17,
10    n_estimators=100,
11    learning_rate=0.1,
12    max_depth=6,
13    subsample=1.0,
14    colsample_bytree=1.0
15 )
16
17 xgb_reg.fit(X_train, y_train)
18 y_valid_pred = xgb_reg.predict(X_valid)
```

The quality of this simple model was evaluated with Mean Absolute Percentage Error (MAPE) and Root Mean Squared Error (RMSE):

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\%, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

For the two feature model I obtained:

$$\text{MAPE} \approx 26.25\%, \quad \text{RMSE} \approx 1562.56.$$

This already shows a reasonable relationship between temperature, wind speed and demand, but the errors are still quite large, which suggests that important calendar and weather effects are not yet captured.

Richer model with calendar and weather features. To improve the model, I extended the feature space with both calendar and additional weather variables.

The following attributes were included:

- **Calendar-based:** season, year, month, holiday flag, weekday, working day, weather situation.
- **Continuous weather variables:** temperature (temp), perceived temperature (atemp), humidity (hum), and wind speed (windspeed).

Categorical variables were transformed using one-hot encoding, while numerical variables were kept as continuous features:

```
1 feature_cols = [  
2     "season", "yr", "mnth", "holiday", "weekday", "workingday",  
3     "weathersit", "temp", "atemp", "hum", "windspeed"  
4 ]  
5  
6 df_model = df[feature_cols + ["cnt"]].copy()  
7  
8 cat_cols = ["season", "yr", "mnth", "holiday",  
9             "weekday", "workingday", "weathersit"]  
10 df_dummies = pd.get_dummies(df_model, columns=cat_cols, drop_first=False)  
11  
12 X_full = df_dummies.drop(columns=["cnt"]).values  
13 y_full = df_dummies["cnt"].values  
14  
15 X_train, X_test, y_train, y_test = train_test_split(  
16     X_full, y_full, test_size=0.3, random_state=17  
17 )
```

The final XGBoost regressor was configured as follows:

```
1 xgb_final = XGBRegressor(  
2     random_state=17,  
3     n_estimators=500,  
4     learning_rate=0.05,  
5     max_depth=6,  
6     subsample=0.8,  
7     colsample_bytree=0.8,  
8     reg_lambda=1.0  
9 )  
10  
11 xgb_final.fit(X_train, y_train)  
12 y_pred = xgb_final.predict(X_test)
```

The test set MAPE for this richer model was:

$$\text{MAPE} \approx 15.44\%,$$

which is a substantial improvement over the two feature baseline (26.25%). Thus, incorporating calendar effects and additional weather variables allows the gradient boosting model to capture much more of the structure in the bike-sharing demand.

Forecast visualization. Figure 34 shows the predicted and actual values for the test set. The model closely follows the day-to-day fluctuations in demand, although some extreme peaks and drops remain challenging due to the inherent variability of user behaviour.

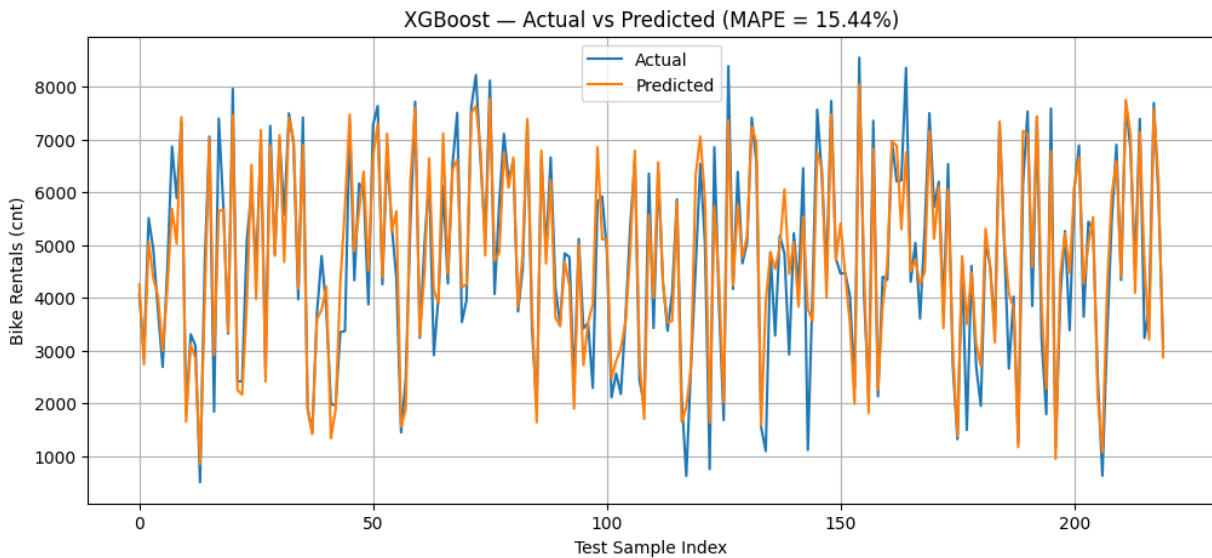


Figure 34: XGBoost predictions vs. actual bike rentals on the test set (MAPE = 15.44%).

Overall, Gradient Boosting demonstrates strong modelling capabilities for bike rental demand. The comparison between the simple two feature model and the richer feature set clearly illustrates how adding informative predictors can significantly reduce forecasting error.

References

- Lakshmi N. Pathi. *Bike Sharing Dataset*. Kaggle. Available at: <https://www.kaggle.com/datasets/lakshmi25npathi/bike-sharing-dataset/data>
- Yury Kashnitsky et al. *mlcourse.ai â Open Machine Learning Course*. Available at: <https://mlcourse.ai/book/index.html>